

Universitatea „Sapientia” din Cluj-Napoca
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
TÎRGU MUREȘ
SPECIALIZAREA CALCULATOARE

Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. univ. dr. Iclănzan David
Andrei

Absolvent:
Nagy Csaba

LUCRARE DE DIPLOMĂ

Coordonator științific:
Conf. univ. dr. Iclănzan David Andrei

Candidat: **Nagy Csaba**
Anul absolvirii: **2026**

a) Tema lucrării de licență:

Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

b) Problemele principale tratate:

- Studiul teoretizat al procesării semnalelor audio și al algoritmilor de învățare profundă (Deep Learning).
- Construirea unui set de date hibrid prin fuziunea a șase baze de date publice (IRMAS, NSynth, Medley-solos-DB, TinySOL, Good-sounds, ESC-50) și înregistrări proprii realizate cu microfon.
- Extragerea și analiza caracteristicilor spectrale: Log-Mel spectrograme, Short-Time Fourier Transform (STFT) și Mel-Frequency Cepstral Coefficients (MFCC).
- Proiectarea, optimizarea și antrenarea unei arhitecturi de rețea neuronală convoluțională bidimensională (2D CNN) în mediul Google Colab.
- Dezvoltarea unui modul de augmentare a datelor (zgomot de fond, reverberație) pentru îmbunătățirea robusteții modelului în condiții acustice reale.
- Validarea sistemului prin implementarea unui modul de recunoaștere în timp real a sunetelor captate la microfon.

c) Desene obligatorii:

d) Softuri obligatorii:

- Prelucrarea setului de date audio în Python 3.
- Proiectarea și antrenarea rețelei neuronale 2D CNN cu TensorFlow și Keras în Google Colab.
- Extragerea caracteristicilor spectrale ale sunetelor utilizând biblioteca Librosa.
- Dezvoltarea modulului de recunoaștere în timp real a sunetelor cu SoundDevice.

e) Bibliografia recomandată:

f) Termene obligatorii de consultații: săptămânal

g) Locul și durata practicii: Universitatea Sapiientia,
Facultatea de Științe Tehnice și Umaniste din Târgu Mureș

Primit tema la data de:

Termen de predare:

Semnătura Director Departament

Semnătura coordonatorului

**Semnătura responsabilului
programului de studiu**

Semnătura candidatului



Declarație

Subsemnatul **Nagy Csaba**, absolvent al specializării **Calculatoare**, promoția **2026** cunoscând prevederile Legii Educației Naționale 1/2011 și a Codului de etică și deontologie profesională a Universității Sapientia cu privire la furt intelectual declar pe propria răspundere că prezenta lucrare de licență/proiect de diplomă/disertație se bazează pe activitatea personală, cercetarea/proiectarea este efectuată de mine, informațiile și datele preluate din literatura de specialitate sunt citate în mod corespunzător.

Târgu Mureș,

Data:

Semnătura:



Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

Extras

Prezenta lucrare de diplomă abordează problema recunoașterii automate a instrumentelor muzicale pe baza înregistrărilor audio, utilizând metode moderne de învățare automată. Sistemul dezvoltat este capabil să clasifice în timp real șapte categorii de instrumente (chitară, pian, voce, instrumente cu coarde, instrumente cu ancie, instrumente de alamă și zgomot ambiental) pornind de la semnale audio monofonice.

Abordarea propusă se bazează pe extragerea a trei tipuri de caracteristici spectrale (Log-Mel spectrograme, STFT și MFCC) și pe antrenarea unor rețele neuronale convoluționale bidimensionale (2D CNN) în mediul Google Colab. Setul de date hibrid a fost creat prin combinarea unor eșantioane curate descărcate de pe internet, a unor re-înregistrări proprii prin microfon și a zgomotului ambiental din baza de date ESC-50, iar un modul de augmentare a datelor simulează condițiile reale de captare prin adăugarea de reverberație, zgomot ambiental și distorsiuni de microfon.

Rezultatele experimentale arată o acuratețe competitivă a modelului pe datele de test, iar sistemul a fost validat prin recunoașterea în timp real a sunetelor captate cu microfonul laptopului. Lucrarea contribuie la demonstrarea fezabilității unui clasificator robust de instrumente muzicale, antrenat cu resurse computaționale accesibile.

Cuvinte cheie: recunoașterea instrumentelor muzicale, rețele neuronale convoluționale, Mel-spectrogramă, STFT, MFCC, clasificare audio, formă de undă, imagine spectrală

Sapientia Erdélyi Magyar Tudományegyetem

MAROSVÁSÁRHELYI KAR

SZÁMÍTÁSTECHNIKA SZAK

Automatikus hangszerfelismerés zenefelvételek alapján

Diplomadolgozat

Témavezető:

**Dr. Iclănzan David Andrei,
egyetemi docens**

Végzős hallgató:

Nagy Csaba

2026

Automatikus hangszerfelismerés zenefelvételek alapján

Kivonat

Jelen diplomadolgozat az automatikus hangszerfelismerés problémáját vizsgálja zenei felvételek alapján, modern gépi tanulási módszerek alkalmazásával. A kidolgozott rendszer valós időben képes osztályozni hét hangszerkategóriát (gitár, zongora, ének, vonós, nádas fúvós, rézfúvós és környezeti zaj) monofón audiojel alapján.

A megvalósított megoldás három spektrális jellemző típust (Log-Mel spektrogram, STFT és MFCC) használ, amelyek kétdimenziós konvolúciós neurális hálózatba (2D CNN) kerülnek betáplálásra. A modell tanítása Google Colab környezetben történt. Az adathalmazt saját tiszta internetes minták és mikrofonnal/telefonnal újrafelvételezett minták, valamint az ESC-50 környezeti zajadatbázis egyesítésével hoztam létre. Az adataugmentációs modul a valós felvételi körülményeket szimulálja visszhang, környezeti zaj és mikrofonkarakterisztika hozzáadásával.

A kísérleti eredmények versenyképes osztályozási pontosságot mutatnak a tesztadatokon, és a rendszert valós idejű mikrofonos felismeréssel is validáltam. A dolgozat hozzájárul annak demonstrálásához, hogy elérhető számítási erőforrásokkal is megvalósítható egy robusztus hangszerosztályozó rendszer.

Kulcsszavak: hangszerfelismerés, konvolúciós neurális hálózat, Mel-spektrogram, STFT, MFCC, audioosztályozás, hullámforma, spektrális kép

Automatic Musical Instrument Recognition Based on Audio Recordings

Abstract

This thesis addresses the problem of automatic musical instrument recognition from audio recordings using modern machine learning methods. The developed system is capable of classifying seven instrument categories in real time (guitar, piano, vocals, strings, reeds, brass, and ambient noise) from monophonic audio signals.

The proposed approach relies on the extraction of three types of spectral features (Log-Mel spectrograms, STFT, and MFCC), which are fed into two-dimensional convolutional neural networks (2D CNN) trained in the Google Colab environment. A hybrid dataset was assembled by combining clean internet-sourced samples, self-recorded microphone playback samples, and ambient noise from the ESC-50 database. A data augmentation module simulates real-world recording conditions by adding reverberation, ambient noise, and microphone characteristics.

Experimental results demonstrate competitive classification accuracy on the test set, and the system has been validated through real-time recognition of sounds captured by a laptop microphone. This work contributes to demonstrating the feasibility of building a robust musical instrument classifier using accessible computational resources.

Keywords: musical instrument recognition, convolutional neural network, Mel-spectrogram, STFT, MFCC, audio classification, waveform, spectral image

Előszó

Amióta az eszemet tudom, a reáltudományok és a művészetek – különösen a zene – világa meghatározó szerepet játszik az életemben. Időről időre az egyik kerül előtérbe, majd a másik veszi át a szerepet, de sosem maradt ki teljesen egyik sem az érdeklődésemből. Ez az oka, hogy sosem tudtam csak egy oldal mellett elköteleződni véglegesen és minden fókuszomat kizárólag ráhelyezni, hogy minél jobb legyek benne. Talán nincs is rá szükség. A reáltudományok segítenek leírni a körülöttem lévő fizikai valóságot és lépést tarthatok vele a technológia exponenciális előretörésével. Ennek ellenére mégis hiányérzetem támad, ha a saját belső világom szeretném megfigyelni. A művészetek körülírják és teljesebbé teszik az ember lelkivilágát, így teremtve meg számomra az egyensúlyt.

Ez az egyensúlykeresés inspirálta jelen kutatásomat és diplomadolgozatomat is, amely a zene és a számítástechnika határterületén valósul meg.

Nagy Csaba

TARTALOMJEGYZÉK

Előszó	iii
Tartalomjegyzék	iv
Ábrák jegyzéke	vi
Táblázatok jegyzéke	viii
1. Bevezető	1
2. Elméleti megalapozás és szakirodalmi tanulmány	3
2.1. Szakirodalmi és technológiai háttér	3
2.2. Digitális hang és hangreprezentáció	4
2.3. Spektrogramok és tanulható bemenetek	5
2.4. Neurális osztályozás spektrogramokon	6
2.5. Hangszer családok akusztikai szemléltetése	7
2.6. Validációs kockázatok: leakage, domain shift, domain adaptation	9
3. A rendszer specifikációi és architektúrája	11
3.1. Feladatpontosítás és rendszerhatárok	11
3.2. Funkcionális követelmények	12
3.3. Nem funkcionális követelmények	12
3.4. Használati esetek	13
3.5. Rendszerarchitektúra és adatfolyam	14
3.6. Validációs és kísérleti protokoll	14
4. Részletes tervezés	16
4.1. Az adathalmaz tervezése	16
4.1.1. A forrás adatok kiválasztása	16
4.1.2. Felosztás és leakage-csökkentés	17
4.1.3. Mikrofonos újrafelvétel	18
4.1.4. A négy kísérleti adathalmaz	19
4.2. Jellemzőkinyerés tervezése	20
4.2.1. A három reprezentáció	20
4.2.2. A megfelelő jellemzőkinyerési eljárás kiválasztása	20
4.2.3. Normalizálás és kimeneti állományok	21
4.3. Adatbővítés tervezése	21
4.4. Modellarchitektúrák tervezése	23
4.4.1. A saját 2D CNN részletei	24
4.4.2. YAMNet transfer learning referencia	25

4.4.3.	Tanítási konfiguráció	25
4.4.3.1.	A tanítás során generált kimenetek	26
4.5.	A valós idejű felismerő modul tervezése	26
4.5.1.	Audio stream és csúszóablak	26
4.5.2.	Normalizálás és stabilizálás	27
4.6.	A feldolgozási folyamat összefoglalása	27
5.	Üzembe helyezés és kísérleti eredmények	29
5.1.	Üzembe helyezési lépések	29
5.2.	Kísérleti protokoll	30
5.3.	A bemeneti reprezentáció kiválasztása a clean baseline alapján	31
5.4.	Validációs eredmények	33
5.5.	Végső teszteredmények	34
5.5.1.	YAMNet transfer learning összehasonlítás	36
5.6.	A jelenlegi rendszer értékelése	38
6.	A rendszer felhasználása	40
6.1.	Az alkalmazás indítása	40
6.2.	Valós idejű feldolgozás futás közben	41
6.3.	A parancssori felület felépítése	41
6.3.1.	Színkódolás és jelölések	42
6.4.	Működési forgatókönyvek	42
6.4.1.	Csendes vagy zajos szobai környezet	42
6.4.2.	Gitárjáték	43
6.4.3.	Emberi énekhang	43
6.4.4.	Nádfúvós és rézfúvós bizonytalanság	44
6.5.	Használati korlátok	44
6.6.	Az alkalmazás leállítása	45
7.	Következtetések	46
7.1.	Megvalósítások	46
7.2.	Hasonló rendszerekkel való összehasonlítás	46
7.3.	Továbbifejlesztési lehetőségek	47
8.	Irodalomjegyzék	48
9.	Függelékek	49
9.1.	A projekt könyvtárszerkezete	49
9.2.	A feldolgozási csővezeték szkriptjei	50
9.3.	Fejlesztői és futtatási környezet	51

ÁBRÁK JEGYZÉKE

2.2.1. A keretezés és ablakozás folyamata: a hangjelet rövid, átfedő ablakokra bontjuk, majd az egyes keretek széleit simítjuk.	5
2.3.1. Egy rövid gitárhang hullámformája és STFT-spektrogramja. A spektrogramon az idő és frekvencia mentén láthatóvá válik, hol található a hang energiája. .	5
2.3.2. Elméleti szemléltetés ugyanazon gitárhang lineáris STFT- és log-Mel spektrogramjával. A Mel-skála az emberi hallás frekvenciaérzékeléséhez igazítja a reprezentációt, ezért a mélyebb tartományt részletesebben kezeli.	6
2.5.1. Kontrollált A4/A3 referenciahangok 20 ms-os nagyított hullámforma-részletei. A cél nem a hangerő, hanem a periodicitás és a jelalak családonkénti szemléltetése.	9
3.4.1. A rendszer fő használati esetei UML jellegű use case ábrán	13
3.5.1. A rendszer magas szintű architektúrája és adatfolyama	14
4.1.1. A saját gyűjtésű tiszta adathalmaz hozzávetőleges eloszlása a szeparált szeleltetés után	18
4.1.2. A mikrofonos újrafelvételi és szinkronizált szeleltelési folyamat	18
4.1.3. A kísérleti adathalmazok hozzávetőleges felépítése osztályonként, kerekített mintaszámokkal	19
4.2.1. A projektben összehasonlított három jellemzőtípus ugyanazon gitárminta esetén: STFT, normalizált Log-Mel és normalizált MFCC	21
4.3.1. A MacBook/mikrofonos felvételi körülményeket közelítő adataugmentációs lánc	22
4.3.2. Reprezentatív minták normalizált Log-Mel spektrogramjai eredeti és augmentált állapotban	23
4.4.1. A saját Log-Mel + 2D CNN modell felépítése	25
4.5.1. A valós idejű Log-Mel + 2D CNN felismerési folyamat	27
4.6.1. A hangszerfelismerő rendszer végső adatfeldolgozási és tanítási folyamata .	28
5.2.1. A kísérleti adathalmazok közelítő megoszlása egy átlagos osztályra vetítve . .	31
5.3.1. A clean baseline validációs Macro F1 értékei a három bemeneti reprezentáció esetén	32
5.4.1. A validációs Macro F1 értékek alakulása a négy kísérleti fázisban	33
5.4.2. A legjobb validációs exp_final futás tanulási görbéi	34
5.5.1. A végső exp_final modell konfúziós mátrixa a teszhalmazon	35
5.5.2. A végső exp_final modell osztályonkénti F1-score értékei	36
5.5.3. A YAMNet transfer learning modell konfúziós mátrixa a teszhalmazon . . .	37

5.5.4.A YAMNet transfer learning modell osztályonkénti F1-score értékei	38
6.1.1.Konzolos példa az alkalmazás indításakor megjelenő konfigurációs adatokra	40
6.4.1.Konzolos példa zaj vagy csend felismerésekor	43
6.4.2.Konzolos példa gitárként azonosított bemenet esetén	43
6.4.3.Konzolos példa vokálként azonosított bemenet esetén	43
6.4.4.Konzolos példa nádfúvós jellegű bemenet esetén	44

TÁBLÁZATOK JEGYZÉKE

2.2.1.A digitális hang legfontosabb alapparaméterei	4
2.3.1.A három vizsgált bemeneti reprezentáció elméleti szerepe	6
2.5.1.A rendszerben használt hangszercsaládok és jellegzetes hullámformáik . . .	8
4.1.1.A vizsgált publikus adathalmazok tervezési szempontú összefoglalása	17
4.1.2.A végleges kísérleti adathalmazok tényleges mérete	19
4.2.1.A három vizsgált jellemzőkinyerési módszer tervezési összehasonlítása . . .	21
4.4.1.A tervezett 2D CNN architektúra rétegei (Log-Mel bemenet: $128 \times 63 \times 1$)	24
4.4.2.A saját 2D CNN tanításához használt fő konfiguráció	26
5.2.1.A négy kísérleti adathalmaz közelítő, osztályonként levezetett mérete	30
5.3.1.Jellemzőkinyerési módszerek összehasonlítása a clean baseline validációs futásain	32
5.4.1.Validációs eredmények öt futás alapján	33
5.5.1.A végső exp_final modell teszteredménye osztályonként	35
5.5.2.A saját Log-Mel modell és a YAMNet transfer learning teszteredményeinek összehasonlítása	37
6.2.1.A valós idejű felismerő modul főbb futásidejű paraméterei	41
6.3.1.A parancssori felületen alkalmazott osztályjelölések	42
9.1.1.A projekt legfelső szintű könyvtárszerkezete	49
9.2.1.A feldolgozási csővezeték szkriptjei és feladataik	50
9.3.1.A fejlesztői környezet főbb összetevői	51

1. BEVEZETŐ

Kodály Zoltán szerint a zene hangzó matematika. Hasonló analógiával élve, a számítástechnika pedig a digitális matematika egy formája, ami számomra azt mutatja meg, hogy valójában mindkét oldal közös tőről fakad.

Kutatásom és ezen dolgozat elkészítése pontosan ebben a metszéspontban, a zene és a számítástechnika határterületén valósul meg. Ezt a közös határterületet a tudományos világban a zenei információ-visszanyerés (Music Information Retrieval – MIR) interdiszciplináris területe képviseli, amely a digitális jelfeldolgozás és a gépi tanulás eszköztárát ötvözi, hogy képessé tegye a számítógépes rendszereket az audiojelek automatikus értelmezésére. Ezen belül az automatikus hangszer- és hangszercsalád-felismerés az egyik legizgalmasabb kutatási irány, amely kulcsfontosságú a digitális zenei adatbázisok intelligens kereshetőségéhez, a valós idejű zeneelemzéshez, valamint az automatikus kottázáshoz (transzkripcióhoz).

Hétköznapi szinten ennek a technológiának az egyik leglátványosabb alkalmazása a modern zenei streaming szolgáltatók (mint például a Spotify vagy az Apple Music) ajánló-rendszereiben érhető tetten. Ezek a platformok kifinomult algoritmusok segítségével elemzik és szabják személyre az általunk preferált zenei mintázatokat, hangszereket és dalokat, megteremtve ezzel a zenehallgatás új, intelligens korszakát. Ennek a személyre szabási folyamatnak a legalapvetőbb lépcsőfokát az olyan alacsony szintű zenei attribútumok és akusztikai jellemzők automatikus felismerése alkotja, mint amilyen a felvételeken megszólaló hangszerek azonosítása is.

A zenében a hangszerek változatos környezetben szólhatnak meg. Háttér szempontjából beszélhetünk „laboratóriumi” stúdiófelvételekről vagy valós idejű, környezeti háttérzajjal terhelt jelekről. A stúdióminőséghez jó analógia a légüres tér, ahol ideális körülmények között csak igencsak minimális zaj található meg, míg normális esetben szinte mindig zaj kíséri a zenét – ha más nem, legalább olyan természeti hangok, mint a szél, a madárcsicsergés vagy a lépések zaja, illetve magának a hangszernek a fizikai megszólaltatása, ami a hangfeldolgozás esetében a második legnagyobb probléma. A hangszerek megszólalhatnak egyedül, de akár többen is egyszerre: több hangszer együttes megszólalása egymásra adódik, úgymond összeolvad, aminek a szétválasztása és megkülönböztetése – mint a hangszerfelismerés „első számú károkozója” – különösen orkesztrák esetén szinte lehetetlen.

A probléma orvoslására az utóbbi időkben a deep learning modellek, különösen a 2D CNN hozott nagy könnyebbiséget, amely a hang szeleteiből hoz létre egy-egy spektrális képet, mintegy a hangok ujjlenyomataként. A hangokat képpé alakítva, a vizuális mintázatok és karakterisztikák alapján történő elemzés lehetővé teszi a hangszerek sajátos színezetének felismerését, ám a szélsőségesen komplex zenekaroknál ez a módszer sem jelent teljes áttörést.

A kutatáshoz használt adathalmazt saját magam állítottam össze az interneten fellelhető zenefelvételek egészen apró szeleteiből, miután alaposan áttekintettem a korábban létező adatbázisokat. Azok a meglévő minták ugyanis nem követték pontosan az én tervezési szándékaimat, és nem adtak választ azokra a specifikus problémákra, amelyeket vizsgálni

szerettem volna. Hosszú távon sokkal célravezetőbb egy teljesen saját, organikus adatbázis felépítése, amely tökéletesen illeszkedik a rendszer egyedi igényeihez.

Az elkészült neurális hálózatra és a saját adathalmazra építve a kutatás végcélja egy olyan alkalmazás kifejlesztése, amely közvetlen, gyakorlati segítséget nyújthat a felhasználóknak. Egy ilyen rendszer továbbfejlesztett változata különösen hasznos lehet azok számára, akik ritkán fordultak meg valós, élő zenei környezetben, és fejletlen a hangszerfelismerési képességük, de gyakran találják magukat olyan helyzetben, amikor egy számukra tetsző, videóklip nélküli dallamban megszólaló hangszer nevét szeretnék azonosítani.

A dolgozat további fejezetei lépcsőről lépésre követik a szoftverfejlesztés lépéseit. A zenei és gépi tanulási elmélet megalapozása után részletesen bemutatásra kerül a rendszer specifikációja és architektúrája, amelyet a 2D CNN hálózat és a valós idejű alkalmazás implementációja, valamint a gyakorlati tesztelés és kiértékelés követ.

2. ELMÉLETI MEGALAPOZÁS ÉS SZAKIRODALMI TANULMÁNY

Ebben a fejezetben azokat az elméleti és szakirodalmi alapokat foglalom össze, amelyek közvetlenül szükségesek a dolgozatban megvalósított hangszerfelismerő rendszer megértéséhez. Nem célom teljes audio-jelfeldolgozási vagy mélytanulási tankönyvet írni; inkább azt mutatom meg, hogy a hang fizikai tulajdonságaiból hogyan jutok el a tanítható spektrogram-reprezentációkig, majd ezekből hogyan lesz osztályozó modell.

2.1. Szakirodalmi és technológiai háttér

Az automatikus hangszerfelismerés több, egymáshoz kapcsolódó területre épül: környezeti hangosztályozásra, zenei információ-visszakeresésre, izolált hangszerhangok elemzésére és neurális audio-reprezentációkra. A saját munkám szempontjából nem az volt a cél, hogy egy meglévő rendszert lemásoljak, hanem az, hogy ezekből a megközelítésekől kiválasszam azt az irányt, amely egy kis méretű, saját gyűjtésű, valós időben is használható rendszerhez illeszkedik.

A környezeti hangosztályozás egyik ismert kiindulópontja az ESC adathalmaz [1], amely rövid hangrészleteken vizsgálja a kategóriefelismerést. Ez a dolgozatban főleg a *noise* osztály miatt fontos: a rendszernek nemcsak hangszercsaládokat kell felismernie, hanem azt is, amikor nincs értelmezhető hangszeres bemenet. Közvetlenebb zenei előzmény az IRMAS adathalmaz [2], amely már kifejezetten hangszerek felismerésére készült, zenei környezetben. Az én rendszerem ennél szűkebb feladatot vállal: rövid, egy másodperces mintákból hangszercsaládot becsül, mikrofonos használati helyzetre készülve.

Az izolált hangszerhangokat tartalmazó adathalmazok közül az NSynth [3], a TinySOL [4], a Medley-solos-DB [5] és a GoodSounds [6] adtak fontos támpontot. Ezek közös tanulsága, hogy a hangszín nem egyetlen egyszerűen mérhető érték, hanem az időbeli burkológörbe, a felharmonikus szerkezet, a megszólaltatási mód és a felvételi környezet együttese. Emiatt a saját rendszeremben nem sok különálló hangszert, hanem hét akusztikailag indokolható osztályt különíték el: gitár, zongora, vokál, vonós, nádfúvós, rézfúvós és zaj/csend.

A megvalósítás gyakorlati alapját Python környezetben a Librosa [7], a NumPy, a Matplotlib és a TensorFlow/Keras [8] adta. A Librosa egységes módon támogatja a hullámformák betöltését, az STFT, MFCC és log-Mel reprezentációk előállítását, míg a TensorFlow/Keras a neurális modellek tanításához és kiértékeléséhez szolgált alapul. Az audio-jelfeldolgozási magyarázatoknál Valerio Velardo *The Sound of AI* oktatási anyagát is felhasználtam [9]. A YAMNet-alapú megközelítés nem a saját pipeline helyettesítője, hanem későbbi transzfer tanulási összehasonlítási pont.

2.2. Digitális hang és hangrepresentáció

A hang fizikai értelemben időben változó nyomáshullám, amelyet a számítógép csak digitális mintasorozatként tud kezelni. A hangszerfelismerés szempontjából négy alapfogalom fontos: az **időtartam**, az **amplitúdó**, a **frekvencia** és a **hangszín**. Az időtartam azt írja le, meddig áll fenn a hang; az amplitúdó a jel kitérésének mértéke; a frekvencia a hangmagasság fizikai alapja; a hangszín pedig az a tulajdonság, amely alapján két azonos hangmagasságú és hangerejű hangot is meg tudunk különböztetni.

A hangszín szempontjából különösen fontos a felharmonikus szerkezet és az időbeli burkológörbe. A természetes hangszerek általában nem egyetlen frekvencián szólnak, hanem az alaphang mellett több felharmonikust is tartalmaznak. Emellett a hang felfutása, kitartása és lecsengése is erősen hangszerfüggő: egy zongorahang hirtelen indul és fokozatosan lecseng, míg egy vonós vagy fúvós hang hosszabban fenntartható.

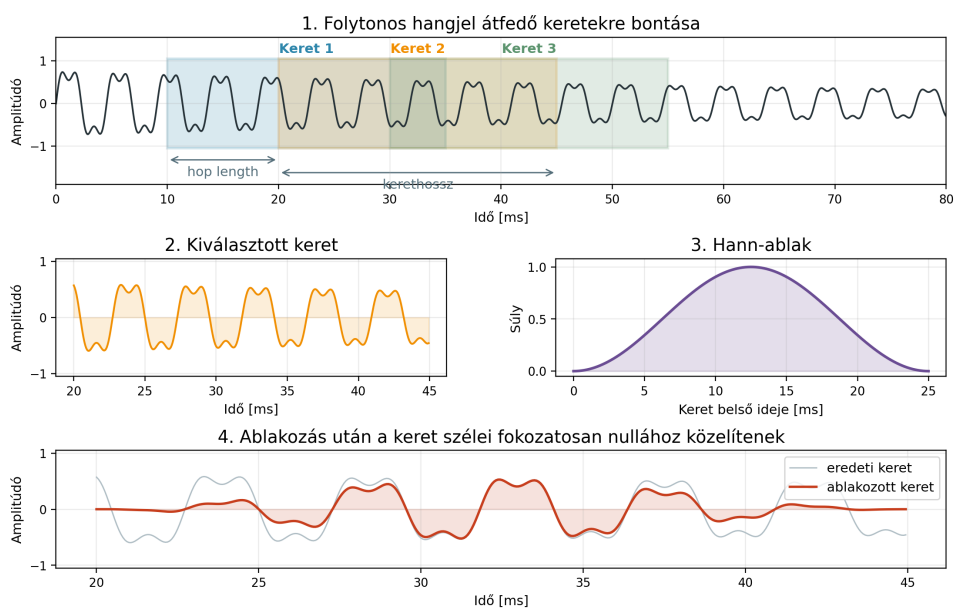
A digitalizálás során az analóg hangból mintavételezéssel és kvantálással lesz diszkrét jelsorozat. A mintavételezési frekvencia azt adja meg, hogy másodpercenként hány ponton mérjük meg a jelet. A Nyquist-tétel alapján a rögzíthető legmagasabb frekvencia a mintavételezési frekvencia fele. A kvantálás az amplitúdóértékeket diszkrét szintekre kerekíti; ennek felbontását a bitmélység határozza meg.

2.2.1. táblázat: A digitális hang legfontosabb alapparaméterei

Paraméter	Példaérték	Jelentés
Mintavételezési frekvencia	16 000 vagy 44 100 Hz	Meghatározza az időbeli felbontást és a rögzíthető felső frekvenciatarományt.
Bitmélység	16 bit	Meghatározza az amplitúdó felbontását és a dinamikatartományt.
Csatornaszám	mono / sztereó	A dolgozatban a feldolgozás egyszerűsítése miatt mono jeleket használók.

A frekvenciaelemzéshez a folytonos mintasorozatot rövid, átfedő keretekre bontjuk. Ezt nevezzük keretezésnek. Mivel a keretek széleinél mesterséges megszakadások keletkezhetnek, az egyes keretekre ablakozó függvényt, például Hann-ablakot alkalmazunk. Ez csökkenti a spektrális szivárgást, vagyis azt a jelenséget, amikor a vágás miatt hamis frekvenciakomponensek jelennek meg.

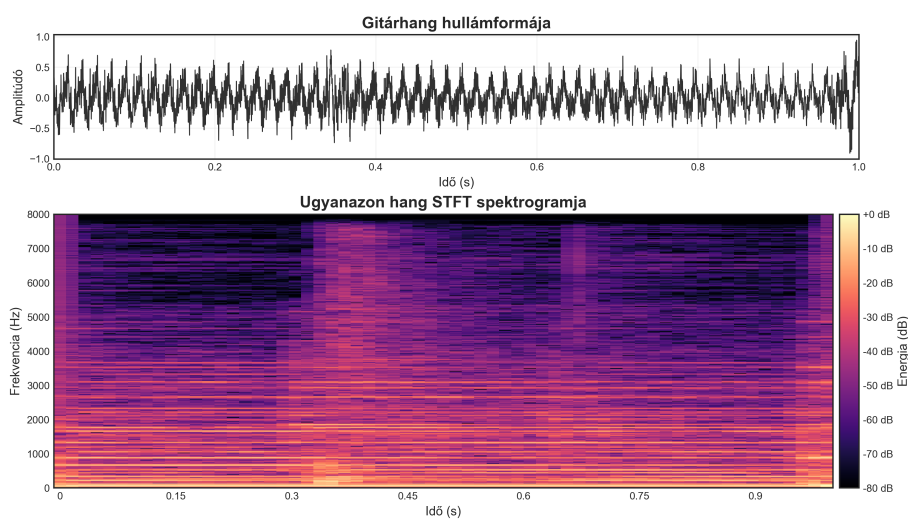
Keretezés és ablakozás audio jelfeldolgozásban



2.2.1. ábra: A keretezés és ablakozás folyamata: a hangjelet rövid, átfedő ablakokra bontjuk, majd az egyes keretek széleit simítjuk.

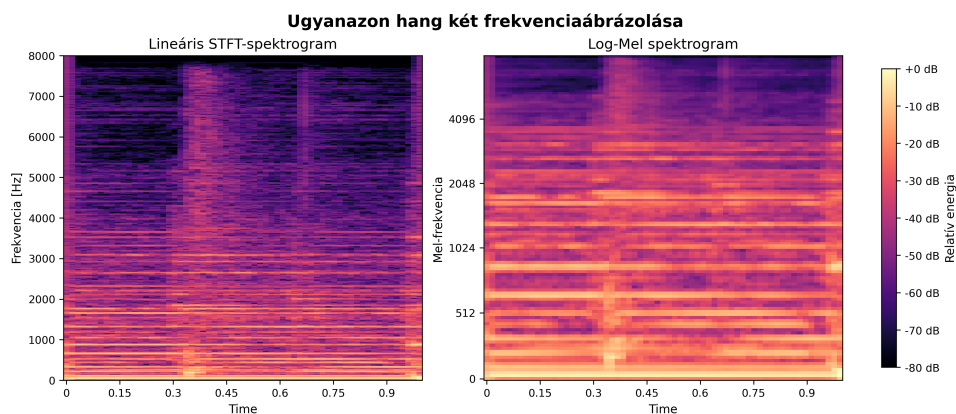
2.3. Spektrogramok és tanulható bemenetek

A nyers hullámforma közvetlenül is tartalmaz minden információt, de egy kis méretű saját modell számára nehezen tanulható bemenet. Ezért a hangot célszerű idő-frekvencia reprezentációvá alakítani. A rövid idejű Fourier-transzformáció (STFT) minden keretet frekvenciakomponensekre bont, az eredmény pedig egy spektrogram: egy képszerű mátrix, ahol az egyik tengely az időt, a másik a frekvenciát, a színintenzitás pedig az energiát jelöli.



2.3.1. ábra: Egy rövid gitárhang hullámformája és STFT-spektrogramja. A spektrogramon az idő és frekvencia mentén láthatóvá válik, hol található a hang energiája.

A lineáris frekvenciaskála mellett gyakran használunk perceptuális skálákat is. Az emberi hallás az alacsony frekvenciák közötti különbségekre érzékenyebb, míg magas frekvenciákon kevésbé finom a megkülönböztetés. Ezt követi a Mel-skála: a mély tartományt részletesebben, a magas tartományt tömörebben ábrázolja. Ha az STFT után Mel-szűrőbankot és logaritmikus decibel skálázást alkalmazunk, log-Mel spektrogramot kapunk.



2.3.2. ábra: Elméleti szemléltetés ugyanazon gitárhang lineáris STFT- és log-Mel spektrogramjával. A Mel-skála az emberi hallás frekvenciaérzékeléséhez igazítja a reprezentációt, ezért a mélyebb tartományt részletesebben kezeli.

A dolgozatban három jellemzőkinyerési irányt használok összehasonlítási alapként. Az **STFT** közvetlen, lineáris frekvenciatengelyű spektrogramot ad, ezért alapvető frekvencia-idő baseline. Az **MFCC** klasszikus, tömörített audiojellemző, amely a Mel-spektrum diszkrét koszinusz transzformációjával keletkezik, és főleg hagyományos gépi tanulási feladatokban terjedt el. A **log-Mel spektrogram** a kettő között helyezkedik el: képszerű struktúrát őriz meg, de a frekvenciatengelyt az emberi halláshoz igazítja.

2.3.1. táblázat: A három vizsgált bemeneti reprezentáció elméleti szerepe

Reprezentáció	Jelleg	Miért került be a dolgozatba?
STFT	Lineáris spektrogram	Megmutatja, hogyan teljesít a közvetlen frekvencia-idő ábrázolás.
MFCC	Tömörített kepsztrális jellemző	Klasszikus audio baseline, kisebb dimenzióval és erősebb előfeldolgozással.
Log-Mel	Perceptuális spektrogram	CNN-hez jól illeszkedő, képszerű és hallásközeleli reprezentáció.

2.4. Neurális osztályozás spektrogramokon

A hangszerfelismerés felügyelt osztályozási feladat: minden tanítómintához tartozik egy bemenet és egy címke. A modell feladata, hogy a bemenet alapján hét kategória közül

válasszon. Többsztályos neurális hálózatonál a kimeneti réteg tipikusan softmax aktivációt használ, amely minden osztályhoz egy valószínűségi értéket rendel. A prediktált osztály az lesz, amelynek a kimeneti értéke a legnagyobb.

A tanítás során a hálózat a hibás predikciók alapján módosítja a súlyait. A validációs halmaz szerepe az, hogy mérje: a modell nemcsak a tanítóadatokat jegyezte-e meg, hanem új mintákon is képes-e általánosítani. A túlilleszkedés (overfitting) akkor jelenik meg, amikor a modell a tanítóhalmazon jól teljesít, de a validációs vagy teszhalmazon gyengébben. Ennek mérséklésére használhatók például dropout rétegek, adataugmentáció, címkesimítás, valamint korai leállítási.

Spektrogramok esetében a 2D konvolúciós neurális hálózat természetes választás, mert a bemenet képszerű: az egyik tengely az időt, a másik a frekvenciát jelöli. A konvolúciós szűrők lokális idő-frekvencia mintázatokat tanulnak, például hirtelen tranzienseket, kitartott harmonikus sávokat vagy zajos spektrális tartományokat.

Nem állítom, hogy a 2D CNN minden alternatívánál általánosan jobb. Egy SVM vagy más klasszikus modell használható lehet kézzel aggregált jellemzővektorokon, egy MLP viszont a spektrogram kilapításával elveszítené a lokális szerkezetet. Az 1D CNN inkább nyers hullámformán vagy időbeli jellemzősorozaton természetes, a CRNN pedig erősebb időbeli modellezést adhat, de összetettebb és több magyarázati terhet jelent. A saját rendszerben a 2D CNN ezért arányos kompromisszum: illeszkedik a log-Mel bemenethez, kezelhető méretű, és valós időben is futtatható.

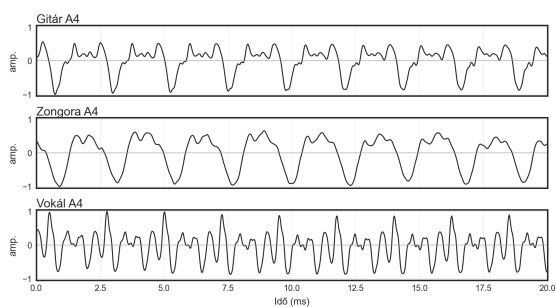
2.5. Hangszerfamiládók akusztikai szemléltetése

A rendszer nem egyedi hangszereket, hanem hangszerfamiládókat különít el. Ez a döntés akusztikailag is indokolható: az egy családba tartozó hangszerek azonos vagy hasonló hangkeltési elvük miatt rokon időbeli és spektrális mintázatokat mutathatnak. A kis adathalmaz és a valós idejű cél miatt ez a csoportosítás védhetőbb, mint sok különálló hangszerosztály kezelése.

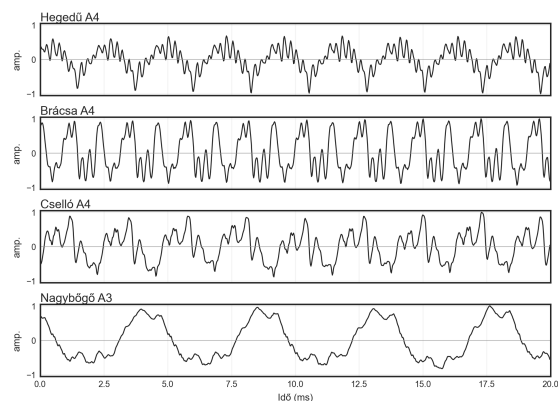
2.5.1. táblázat: A rendszerben használt hangszercsaládok és jellegzetes hullámformáik

Osztály	Példák	Hangkeltés	Jelleg	Hullámforma
Gitár	Akusztikus, elektromos gitár	Húr pengetése	Gyors attack, majd lecsengő rezgés.	
Zongora	Zongora, e-piano	Kalapács által leütött húr	Impulzusszerű kezdet, hosszabb lecsengés.	
Vokál	Énekhang, torokének	Hangszálak rezgése	Kváziperiodikus jel, formánsokkal.	
Vonós	Hegedű, brácsa, cselló, nagybőgő	Vonó és húr súrlódása	Kitartott, folytonos gerjesztés.	
Nádfúvós	Klarinét, szaxofon, oboa, fagott	Nád rezgése	Kitartott, periodikus, rezonáns jel.	
Rézfúvós	Trombita, harsona, kürt, tuba	Ajkak rezgése fúvókán	Erős harmonikus szerkezet, nagy energia.	
Zaj/Csend	Környezeti zajok	Véletlenszerű rezgések	Aperiodikus, rendszertelen jel.	

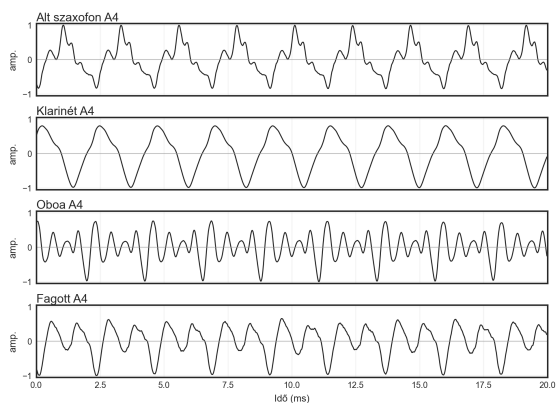
A táblázatban hosszabb hullámforma-részletek szerepelnek, mert ezeken jól látszik a hang felfutása és lecsengése. Emellett kontrollált, közeli példákat is készítettem az asd/ mappában található hangmintákból. A legtöbb hangszer esetében A4 hangot használtam, a nagybőgő és a tuba esetében A3-at, mert ezeknél ez természetesebb regiszter. Ezek az ábrák nem tanítóadatok, hanem szemléltető példák: a periódus belső alakjára és a családon belüli hasonlóságokra nagyítanak rá.



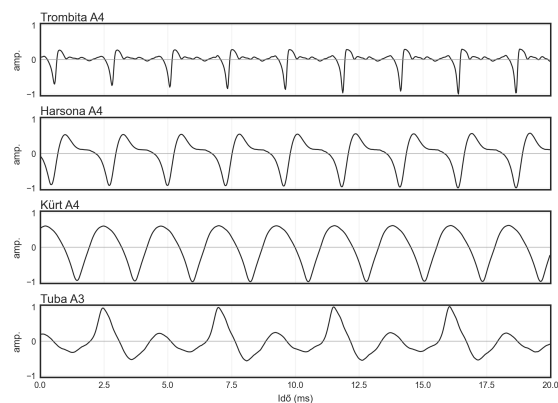
(a) Gitár, zongora és vokál



(b) Vonós hangszerek



(c) Nádfűvös hangszerek



(d) Rézfűvös hangszerek

2.5.1. ábra: Kontrollált A4/A3 referenciahangok 20 ms-os nagyított hullámforma-részletei. A cél nem a hangerő, hanem a periodicitás és a jelalak családonkénti szemléltetése.

2.6. Validációs kockázatok: leakage, domain shift, domain adaptation

Gépi tanulási rendszereknél a kiértékelés egyik legfontosabb kockázata az adatszivárgás, vagyis amikor a modell a tanítás során olyan információhoz jut, amely a valós használat során nem lenne elérhető. Audio esetén ez különösen könnyen előfordulhat, mert egy hosszabb felvétel egymáshoz közeli szeletei nagyon hasonlóak lehetnek. Ha ezek egyszerre kerülnek train és test halmazba, a modell túl optimista eredményt adhat.

A dolgozat szempontjából három leakage-típust érdemes kiemelni. A **train-test kontamináció** akkor jelenik meg, ha nagyon hasonló vagy azonos minták kerülnek különböző splitbe. Az **előfeldolgozási leakage** akkor fordul elő, ha például normalizálási paramétereket a teljes adathalmazból számolunk, majd azokat a tesztadatokra is alkalmazzuk. Az **augmentációs leakage** pedig akkor veszélyes, ha egy eredeti minta az egyik, annak torzított változata pedig egy másik halmazba kerül.

A valós idejű mikrofonos felismerésnél emellett megjelenik a **domain shift** is. Ez azt jelenti, hogy a tanítási környezet és a célkörnyezet eloszlása eltér: a tiszta internetes vagy stúdiószerű hangok másképp viselkednek, mint a telefonról visszajátszott és laptopmikrofonnal

rögzített minták. A **domain adaptation** ennek kezelésére szolgál: a célkörnyezetből származó mikrofonos train minták bevonásával a modell jobban alkalmazkodhat ahhoz az akusztikai helyzethez, amelyben később működnie kell.

Ezek miatt a későbbi kísérletekben a train, validation és test halmazokat külön szerepben kezelem. A validáció a modellválasztásra szolgál, a test halmaz pedig csak a kiválasztott végső modellek utolsó ellenőrzésére. Így a mérési eredmények nem rekordpontosságként, hanem kontrollált, módszertanilag védhető összehasonlításként értelmezhetők.

3. A RENDSZER SPECIFIKÁCIÓI ÉS ARCHITEKTÚRÁJA

Az előző fejezetben bemutatott akusztikai és gépi tanulási alapok után ebben a fejezetben pontosítom, hogy milyen rendszert kellett megterveznem. Itt még nem a konkrét rétegek, hiperparaméterek vagy mérési eredmények a fontosak, hanem az, hogy a projekt milyen feladatot old meg, milyen határok között értelmezhető, és milyen fő komponensekből áll.

A célom egy olyan hangszerfelismerő rendszer készítése volt, amely előkészített hangmintákon tanítható, több kísérleti adathalmaz-változaton ellenőrizhető, majd a kiválasztott saját modell valós időben, mikrofonról is futtatható. A rendszer központi megoldása a saját Log-Mel spektrogramra épülő 2D CNN pipeline; a YAMNet transfer learning modell összehasonlító referencia, nem pedig a valós idejű alkalmazás fő útvonala.

3.1. Feladatpontosítás és rendszerhatárok

A dolgozat feladata hét hangosztály felismerése rövid, 1 másodperces hangminták alapján. A hat hangszeres kategória a gitár, zongora, vokál, vonós, nádfúvós és rézfúvós, ezeket egészíti ki a zaj/csend kategória. A választott felbontás szándékosan hangszercsalád-szintű: nem az a cél, hogy például a trombitát és a harsonát minden esetben különválasszam, hanem hogy a rendszer egy valós időben is kezelhető, robusztusabb kategóriarendszeren működjön.

A rendszer határait a következőképpen határoztam meg:

- **Bemenet:** 16 kHz-re egységesített, mono hanganyag, offline tanításhoz fájlokból, valós idejű használathoz mikrofonból.
- **Kimenet:** egy hét elemű valószínűségi vektor, illetve a legvalószínűbb hangosztály megjelenítése.
- **Fő felhasználás:** kutatási és demonstrációs célú hangszerfelismerés, külön hangsúlyt adva a mikrofonos célkörnyezet hatásának.
- **Nem cél:** polifón hangszer-szétválasztás, pontos hangszerazonosítás egy családon belül, mobilalkalmazás vagy ipari minőségű zenei elemző rendszer készítése.

Ez a lehatárolás azért fontos, mert a rendszer minőségét nem egy általános zenei AI-platform mércéjével kell értelmezni. A dolgozat fő értéke a teljes adatfolyam megtervezése: saját adathalmaz, augmentáció, mikrofonos domain shift, modellválasztás, validáció, végső teszt és valós idejű működés.

3.2. Funkcionális követelmények

A funkcionális követelmények azt írják le, hogy a rendszernek milyen műveleteket kell ténylegesen elvégeznie. Ezeket a követelményeket úgy határoztam meg, hogy lefedjék a teljes kutatási és alkalmazási folyamatot az adatgyűjtéstől az élő predikcióig.

1. **Tiszta adathalmaz előállítása:** a rendszer képes legyen az internetes forrásokból válogatott hangszerez mintákat és az ESC-50-ből származó zajmintákat egységes szerkezetbe rendezni.
2. **Train/validation/test szeparáció:** a felosztás még a rövid, 1 másodperces szeletek létrehozása előtt történjen, hogy az egymáshoz közeli szeletek ne kerüljenek külön halmazokba.
3. **Mikrofonos újrafelvétel kezelése:** a tiszta mintákból split szerint rendezett, visszajátzott és újrarögzített mikrofonos változatok készülhessenek.
4. **Kísérleti adathalmazok összeállítása:** a rendszer külön kezelje a clean, augmented, naive deployment és final kísérleti adathalmazokat.
5. **Jellemzőkinyerés:** a rendszer állítson elő Log-Mel, STFT és MFCC reprezentációkat, valamint a YAMNethez szükséges nyers hanghullámot ott, ahol ez indokolt.
6. **Modelltanítás és riportolás:** a tanítás mentse a legjobb checkpointot, a classification reportot, a konfúziós mátrixot és a fontosabb diagramokat úgy, hogy az ismételt futások ne írják felül egymást.
7. **Valós idejű felismerés:** a kiválasztott saját Log-Mel CNN modell legyen betölthető mikrofonos predikcióhoz, simítással és küszöböléssel stabilizált terminálos visszajelzéssel.

3.3. Nem funkcionális követelmények

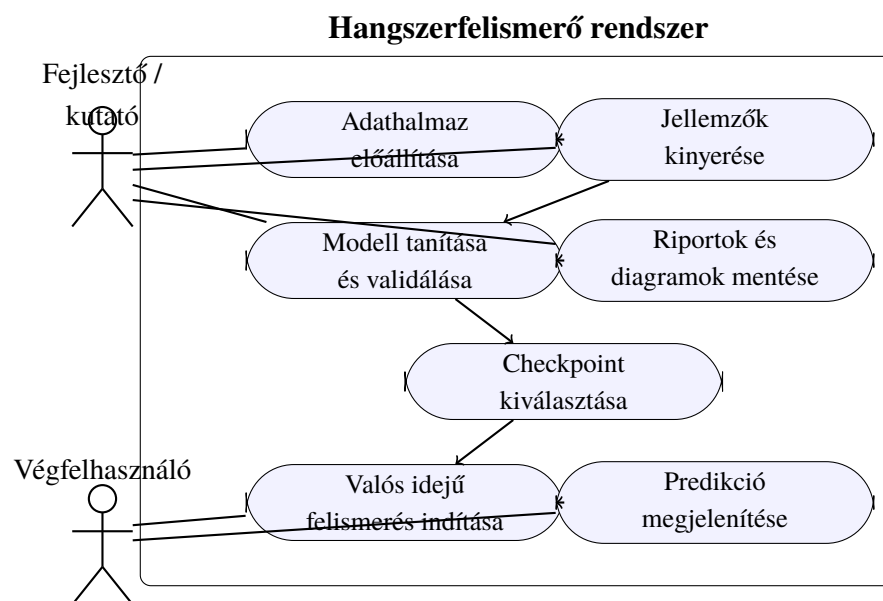
A nem funkcionális követelmények a rendszer minőségi tulajdonságait határozzák meg. Ezek a dolgozat szempontjából legalább olyan fontosak, mint maga a predikciós pontosság, mert egy gépi tanulási rendszer könnyen adhat látványos, de módszertanilag gyenge eredményeket.

- **Reprodukálhatóság:** a szkriptekben a véletlen műveletekhez rögzített seedet használok, és az eredményfájlok időbélyeges névvel készülnek.
- **Adatszivárgás csökkentése:** a felosztás blokkszinten történik, az augmentáció csak a tanítóhalmazon jelenik meg, és a végső tesztet csak a kiválasztott modellek ellenőrzésére használom.

- **Domain shift mérhetősége:** külön validációs és tesztelési helyzetben szerepel a mikrofonos domain, hogy látható legyen a tiszta és a célkörnyezeti adat közötti különbség.
- **Erőforrás-hatékonyság:** a saját modell mérete maradjon kezelhető, hogy a betanított hálózat egy átlagos laptopon is használható legyen valós időben.
- **Modularitás:** az adatelőkészítés, jellemzőkinyerés, tanítás és valós idejű felismerés külön futtatható egységekre legyen bontva.
- **Átlátható kiértékelés:** az eredmények ne csak egyetlen accuracy értéként jelenjenek meg, hanem osztályonkénti precision, recall, F1-score és konfúziós mátrix formájában is.

3.4. Használati esetek

A rendszer két fő használati mód köré szerveződik. Az első a fejlesztői/kutatási folyamat, ahol az adathalmazok előkészítése, a jellemzőkinyerés és a modellek tanítása történik. A második a végfelhasználói jellegű valós idejű használat, ahol a korábban kiválasztott modell mikrofonból érkező hangra ad visszajelzést.

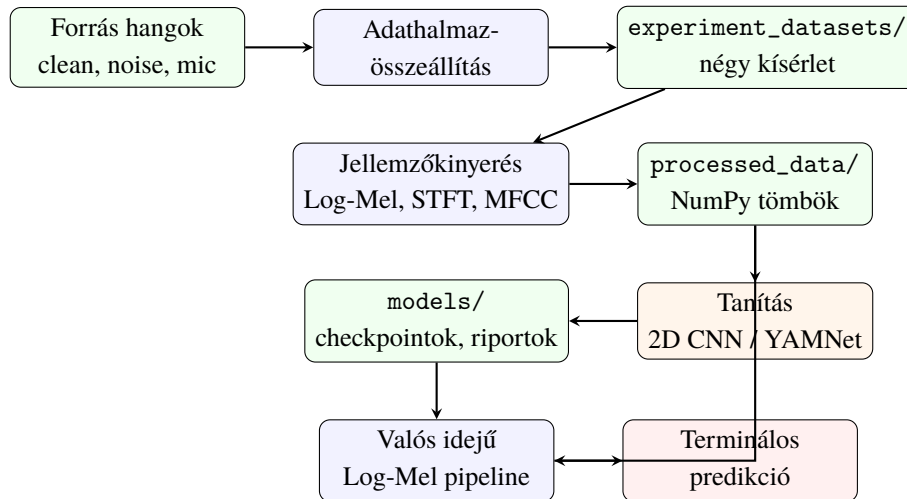


3.4.1. ábra: A rendszer fő használati esetei UML jellegű use case ábrán

Az ábra szándékosan nem különíti el minden szkript apró részfeladatát. A specifikáció szintjén az a lényeges, hogy a rendszernek legyen egy kutatási ága, amelyből hitelesen kiválasztható egy modell, és legyen egy alkalmazási ága, amely ezt a modellt élő hangon használja.

3.5. Rendszerarchitektúra és adatfolyam

A rendszer architektúráját adatfolyamként érdemes értelmezni. A nyers hangfájlokból először kísérleti adathalmazok készülnek, ezekből jellemzőmátrixok, majd ezekre épülnek a tanított modellek. A valós idejű felismerés ugyanazt a Log-Mel reprezentációt használja, mint a tanítás, így a tanító és az alkalmazási útvonal között nincs módszertani törés.



3.5.1. ábra: A rendszer magas szintű architektúrája és adatfolyama

A 3.5.1. ábra alapján a rendszer három nagy részre bontható. Az első az adatelőkészítő rész, amely a nyers hangokból kontrollált kísérleti adathalmazokat állít elő. A második a tanítási és kiértékelési rész, ahol a modellek és riportok keletkeznek. A harmadik az alkalmazási rész, amely a kiválasztott checkpointot betölti, és mikrofonból érkező hangon futtatja a predikciót.

3.6. Validációs és kísérleti protokoll

A mérési protokollt úgy alakítottam ki, hogy a modellválasztás és a végső ellenőrzés elkülönüljön. A tanítás és a modellválasztás validációs halmazon történik; a test halmazt csak a kiválasztott végső modellek lezáró ellenőrzésére használom. Ez különösen fontos kis adathalmaznál, mert ha a test eredmény visszahatna a modellválasztásra, akkor a dolgozat túl optimista képet adna a generalizációról.

A négy kísérleti adathalmaz szerepe a következő:

- **exp_clean:** tiszta train, tiszta validáció és tiszta test; ez az alap baseline.
- **exp_augmented:** tiszta és augmentált train, tiszta validáció és tiszta test; ez az augmentáció hatását méri.
- **exp_naive_deployment:** tiszta és augmentált train, de mikrofonos validáció és test; ez mutatja a domain shiftet.

- `exp_final`: tiszta, augmentált és mikrofonos train, mikrofonos validáció és test; ez a végső célrendszer.

Ezzel a felépítéssel nem csak az látható, hogy egy modell tiszta adatokon hogyan teljesít, hanem az is, hogy mi történik, amikor a bemenet átkerül a célkörnyezetbe. A részletes adathalmaz-tervezést, feature-választást és modellarchitektúrát a következő fejezet tárgyalja.

4. RÉSZLETES TERVEZÉS

Ebben a fejezetben a rendszer konkrét tervezési döntéseit mutatom be. A specifikációs fejezetben meghatározott követelmények után itt már az a kérdés, hogyan épül fel a saját adathalmaz, milyen jellemzőket érdemes kinyerni a hangból, hogyan készülnek a kísérleti adathalmazok, és milyen modellarchitektúra illeszkedik ehhez a feladathoz.

A fejezet nem minden egyes segédfüggvény dokumentációja. Inkább azt a mérnöki gondolatmenetet követi végig, amely elvezetett a végső Log-Mel + 2D CNN pipeline-hoz, valamint a YAMNet transfer learning összehasonlító modellhez.

4.1. Az adathalmaz tervezése

A hangszerfelismerési feladatban az adathalmaz minősége legalább annyira meghatározó, mint maga a neurális hálózat. A célom nem egy nagyon nagy, de nehezen átlátható adathalmaz létrehozása volt, hanem egy kisebb, kontrollált, kísérletezésre alkalmas adatbázis, amelyben a tiszta, augmentált és mikrofonos változatok szerepe elkülöníthető.

Az osztályrendszert hét kategóriára bontottam:

- (1) Gitár: akusztikus és kevésbé torzított elektromos gitárok,
- (2) Zongora: klasszikus és digitális zongorahangok,
- (3) Vokál: énekhang és néhány karakteres emberi hangképzés,
- (4) Vonós: hegedű, brácsa, cselló, nagybőgő,
- (5) Nádfúvós: klarinét, szaxofon, oboa, fagott,
- (6) Rézfúvós: trombita, harsona, kürt, tuba,
- (7) Zaj/csend: környezeti zajok, háttérzajok és csendközeli szakaszok.

Ez a kategóriarendszer tudatos kompromisszum. A rövid, 1 másodperces mintákon belül egyes konkrét hangszerek nagyon hasonló akusztikai jegyeket hordoznak, ezért a hangszercsalád-szintű felismerés stabilabb és a valós idejű demonstrációhoz is jobban illeszkedik.

4.1.1. A forrásadatok kiválasztása

A tervezés elején megvizsgáltam néhány ismert publikus adathalmazt, mert ezek jól mutatják, milyen irányok léteznek a hangszerfelismerésben. A cél végül nem ezek összeolvasztása lett, hanem egy saját, a dolgozat céljához igazított organikus adathalmaz létrehozása. A vizsgált források legfontosabb tanulságait a 4.1.1. táblázat foglalja össze.

4.1.1. táblázat: A vizsgált publikus adathalmazok tervezési szempontú összefoglalása

Adathalmaz	Tartalom	Hasznos tanulság	Korlát ebben a projektben
IRMAS [2]	Valós zenei részletek szó-ló hangszeres jelöléssel	Jó példa valós zenei környezetre	Eltérő kategóriarendszer és korlátozott egyensúly
NSynth [3]	Nagy mennyiségű izolált hangszerhang	Egységes, kontrollált minták	Túl steril; nem folytonos zenei részletek
TinySOL [4]	Izolált zenekari hangszerek	Jó minőségű referenciahangok	Kevésbé reprezentálja a valós előadást
Good-sounds [6]	Fúvós és vonós hangjegyek	Változatos felvételi minőségek	Hiányzik több célosztály, például gitár és vokál
ESC-50 [1]	Környezeti hangok és zajok	Kiváló zaj/csend kategóriához	Nem tartalmaz hangszeres osztályokat

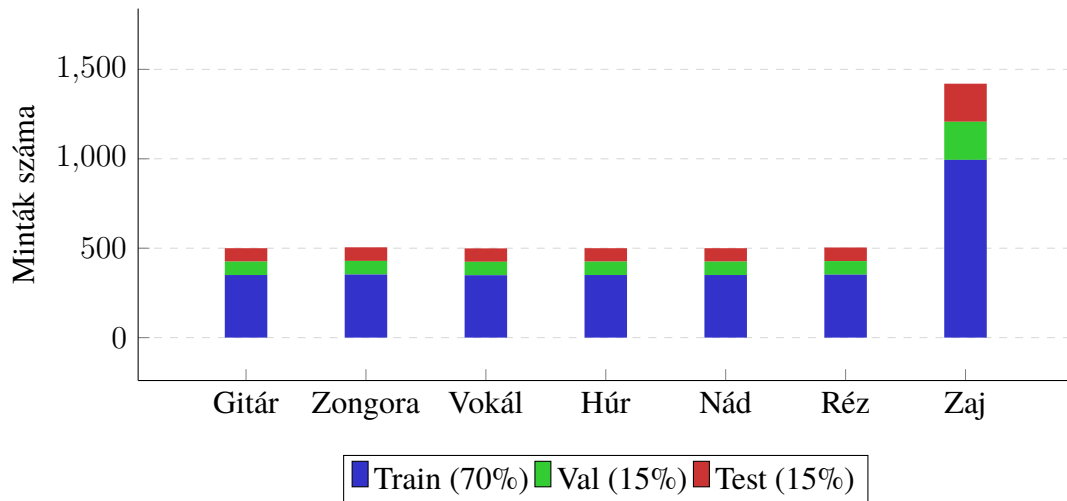
A végső döntés az lett, hogy a hangszeres osztályokhoz interneten elérhető zenei felvételekből válogatok karakteres, többnyire jól felismerhető részleteket, a zaj/csend osztályt pedig az ESC-50 adathalmazból egészítem ki. Az internetes hangmintákat kizárólag oktatási és kutatási célra használom, a nyers forrásfájlokat nem teszem közzé és nem terjesztem.

4.1.2. Felosztás és leakage-csökkentés

A felosztásnál a legfontosabb tervezési kockázat az adatszivárgás volt. Ha először 1 másodperces szeleteket hoznék létre, és csak ezután osztanám szét őket train, validation és test halmazba, akkor ugyanannak az 5 másodperces zenei blokknak nagyon hasonló részletei több halmazba is bekerülhetnének. Ez túl optimista validációs eredményt adna.

Ezért a `01_prepare_clean_dataset.py` először az 5 másodperces blokkokat osztja szét rétegzett, osztályonként kiegyensúlyozott módon, nagyjából 70%/15%/15% arányban. Az 1 másodperces szeletelés csak ezután történik meg. Így az egy blokkból származó szeletek azonos halmazban maradnak. Forrásfelvétel-szintű teljes szeparációt a kis adatmennyiség miatt nem állítok, de a szeletek közötti közvetlen leakage kockázatát ezzel a megoldással jelentősen csökkentettem.

Az első, tiszta adathalmaz hozzávetőleges eloszlását a 4.1.1. ábra szemlélteti.

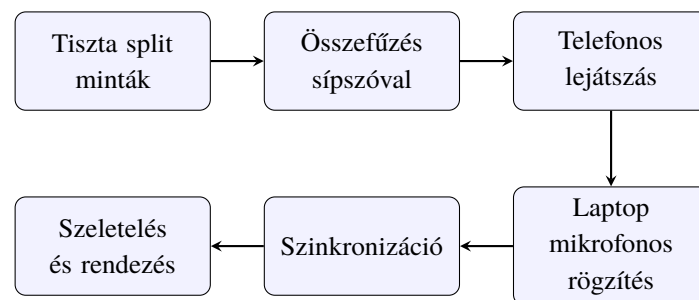


4.1.1. ábra: A saját gyűjtésű tiszta adathalmaz hozzávetőleges eloszlása a szeparált szeletelés után

4.1.3. Mikrofonos újrafelvétel

A projekt egyik fontos célja a valós idejű, mikrofonos felismerés. Emiatt a tiszta internetes minták önmagukban nem elegendőek, mert nem tartalmazzák a saját laptopmikrofon, a szoba és a visszajátszó eszköz hatását. Ezt a különbséget domain shiftnek tekintem, és külön kísérleti adathalmazokkal mérem.

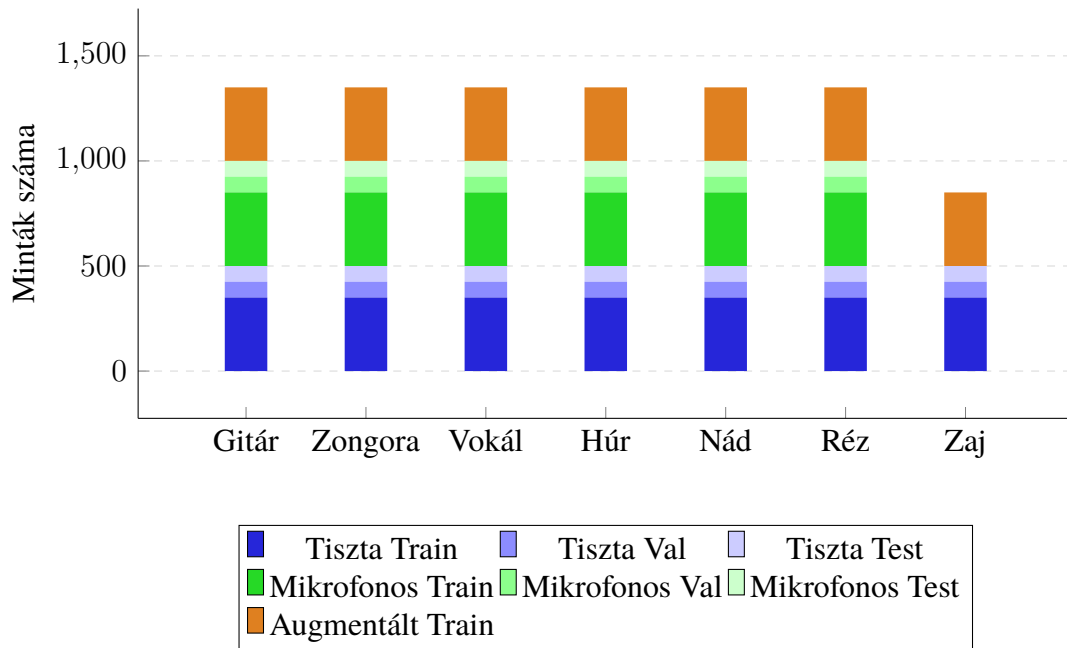
A mikrofonos adatok előállításához nem egyenként játszottam fel minden mintát, hanem automatizált folyamatot terveztem. A `02_acquire_mic_data.py` split és osztály szerint összefűzi a tiszta mintákat, a hanganyag elejére szinkronizációs sípszót tesz, majd a telefonról visszajátszott jelet a laptop beépített mikrofonjával rögzíti. A script a sípszó alapján újraigazítja a felvételt, és visszaszeteleli 1 másodperces mintákra.



4.1.2. ábra: A mikrofonos újrafelvételi és szinkronizált szeletelési folyamat

A mikrofonos train, validation és test változatok szerepe elkülönül. A mikrofonos train minták csak a végső modell tanításába kerülnek be, míg a mikrofonos validation és test minták a célkörnyezet mérésére szolgálnak. Így nem tanítok rá a kiértékelési mintákra, de mégis meg tudom mérni, mennyit változik a teljesítmény a mikrofonos domainben.

A kísérleti adathalmazok egy osztályra vetített, kerekített felépítését a 4.1.3. ábra mutatja.



4.1.3. ábra: A kísérleti adathalmazok hozzávetőleges felépítése osztályonként, kerekített mintaszámokkal

4.1.4. A négy kísérleti adathalmaz

A végső adatfolyam nem egyetlen nagy adathalmazt hoz létre, hanem négy elkülönített kísérleti változatot az `experiment_datasets/` könyvtárban. Ezekből a `04a_extract_features.py` script készíti el a tanításhoz használt NumPy tömböket a `processed_data/` könyvtár alatt.

4.1.2. táblázat: A végleges kísérleti adathalmazok tényleges mérete

Adathalmaz	Train	Validation	Test
exp_clean	2420	542	536
exp_augmented	4841	542	536
exp_naive_deployment	4841	562	557
exp_final	7278	562	557

Az `exp_clean` az alap tiszta mérés, az `exp_augmented` a szoftveres augmentáció hatását mutatja, az `exp_naive_deployment` a mikrofonos célkörnyezet okozta visszaesést méri, az `exp_final` pedig a végső, mikrofonos train adatokkal is megerősített rendszert képviseli. A validációs és teszhalmaz szerepe itt is elkülönül: validáció alapján választok modellt, a test csak a kiválasztott végső modellek ellenőrzésére szolgál.

4.2. Jellemzőkinyerés tervezése

A nyers hullámforma közvetlenül is használható mélytanulási bemenetként, de a saját 2D CNN modellhez célszerűbb idő-frekvencia reprezentációt készíteni. A tervezés során három, szakmailag jól elkülöníthető jellemzőtípust vizsgáltam:

- **STFT spektrogram:** lineáris frekvenciaskálájú, közvetlen idő-frekvencia ábrázolás;
- **MFCC:** klasszikus, tömörített, kepsztrális audio baseline;
- **Log-Mel spektrogram:** perceptuális, képszerű reprezentáció, amely jól illeszkedik a 2D CNN-hez.

Nem az volt a célom, hogy minden lehetséges zenei jellemzőt kipróbáljak. A chroma, tonnetz vagy spectral contrast inkább hangmagassági és harmóniai jellegű feladatokban hasznos, míg itt rövid minták hangszercsalád-szintű osztályozása a cél. Az STFT–MFCC–Log-Mel hármas ezért elég széles összehasonlítási alapot ad: nyersebb spektrum, klasszikus tömörítés és perceptuális spektrogram.

A használt közös beállítások:

- mintavételi frekvencia: $SR = 16\,000$ Hz,
- FFT ablakméret: $n_{fft} = 1024$,
- Log-Mel hop length: 256 minta,
- STFT és MFCC összehasonlító hop length: 512 minta.

4.2.1. A három reprezentáció

STFT. Az STFT decibelre konvertált magnitúdó-spektrogramot ad, 513×32 méretben. Előnye, hogy megtartja a teljes lineáris frekvenciatengelyt, hátránya viszont a nagyobb bemeneti méret és a kevésbé perceptuális frekvenciafelbontás.

MFCC. Az MFCC-t 40 együtthatóval számolom, így a bemenet 40×32 méretű. Ez nagyon kompakt reprezentáció, de a diszkrét koszinusz transzformáció miatt eldobhat olyan spektrális finomszerkezeteket, amelyek a hangszerek hangszínéhez fontosak.

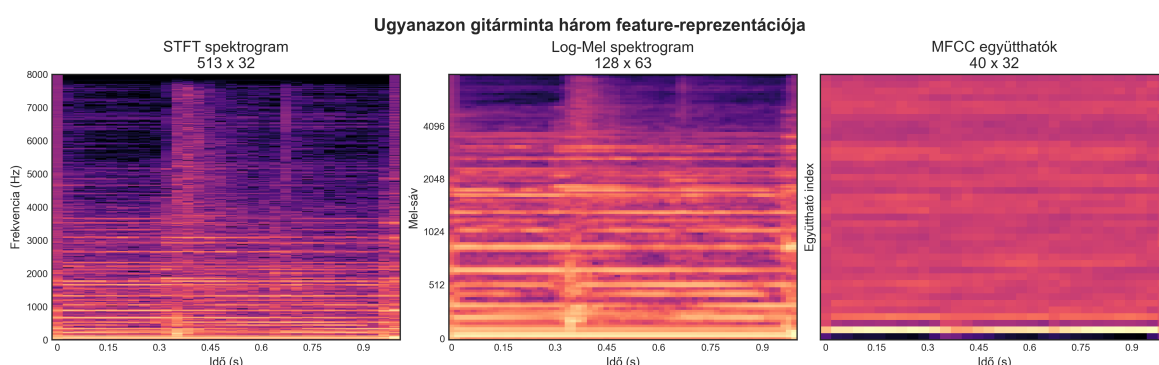
Log-Mel. A Log-Mel spektrogram 128 Mel-sávot használ, a finomabb hop length miatt pedig 128×63 méretű mátrixot ad. Ez lett a saját modell fő bemenete, mert a hangszínt hordozó spektrális szerkezetet még jól megtartja, miközben a bemenet kezelhető méretű marad.

4.2.2. A megfelelő jellemzőkinyerési eljárás kiválasztása

A döntést két szempont vezette. Egyrészt a Log-Mel elméletileg is illeszkedik a hallásközel, képszerű reprezentációhoz; másrészt a clean baseline kísérletekben ez bizonyult a legerősebb saját CNN bemenetnek. Emiatt a későbbi augmentációs, domain shift és final kísérleteket már Log-Mel bemenettel futtattam.

4.2.1. táblázat: A három vizsgált jellemzőkinyerési módszer tervezési összehasonlítása

Jellemző	Dimenzió	Skála	Fő előny	Fő korlát
STFT	513×32	Lineáris	Megőrzi a teljes spektrális szerkezetet.	Nagyobb bemenet, sok magas frekvenciás részlet.
Log-Mel	128×63	Mel-skála	Hallásközele, CNN-hez jól illeszkedő reprezentáció.	A Mel-sűrítés enyhe információvesztést okoz.
MFCC	40×32	Kepsztrális	Nagyon kompakt klasszikus baseline.	Kevésbé őrzi meg a felharmonikus mintázatot.



4.2.1. ábra: A projektben összehasonlított három jellemzőtípus ugyanazon gitárminta esetén: STFT, normalizált Log-Mel és normalizált MFCC

4.2.3. Normalizálás és kimeneti állományok

A Log-Mel és STFT jellemzőket decibel skálára alakítom, majd a tanítás és a valós idejű felismerés között konzisztens módon $[0, 1]$ tartományra skálázom. Az MFCC esetében sávonkénti standardizálást használok. A lényeg az, hogy a modell ugyanabban az értéktartományban kapjon bemenetet tanításkor és inferencia közben.

A kinyert tömbök a `processed_data/` könyvtár alá kerülnek, kísérleti adathalmazként külön mappába. A Log-Mel minden kísérlethez elkészül, az STFT és MFCC csak a clean baseline feature-összehasonlításhoz szükséges, a nyers hanghullám pedig az `exp_final` YAMNet transfer learning méréséhez készül el.

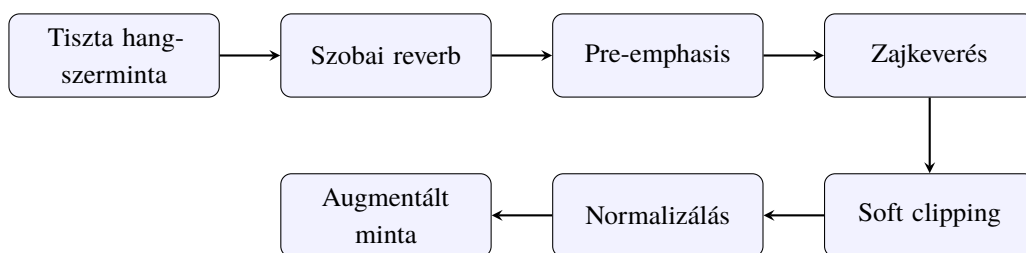
4.3. Adatbővítés tervezése

A szoftveres augmentáció célja az volt, hogy a tiszta train minták mellé olyan torzított változatok kerüljenek, amelyek valamennyire hasonlítanak egy fogyasztói eszközön visszajátzott és laptopmikrofonnal rögzített hanghoz. Az augmentáció csak a tanítóhalmazon történik; a validációs és teszhalmazt nem torzítom utólag.

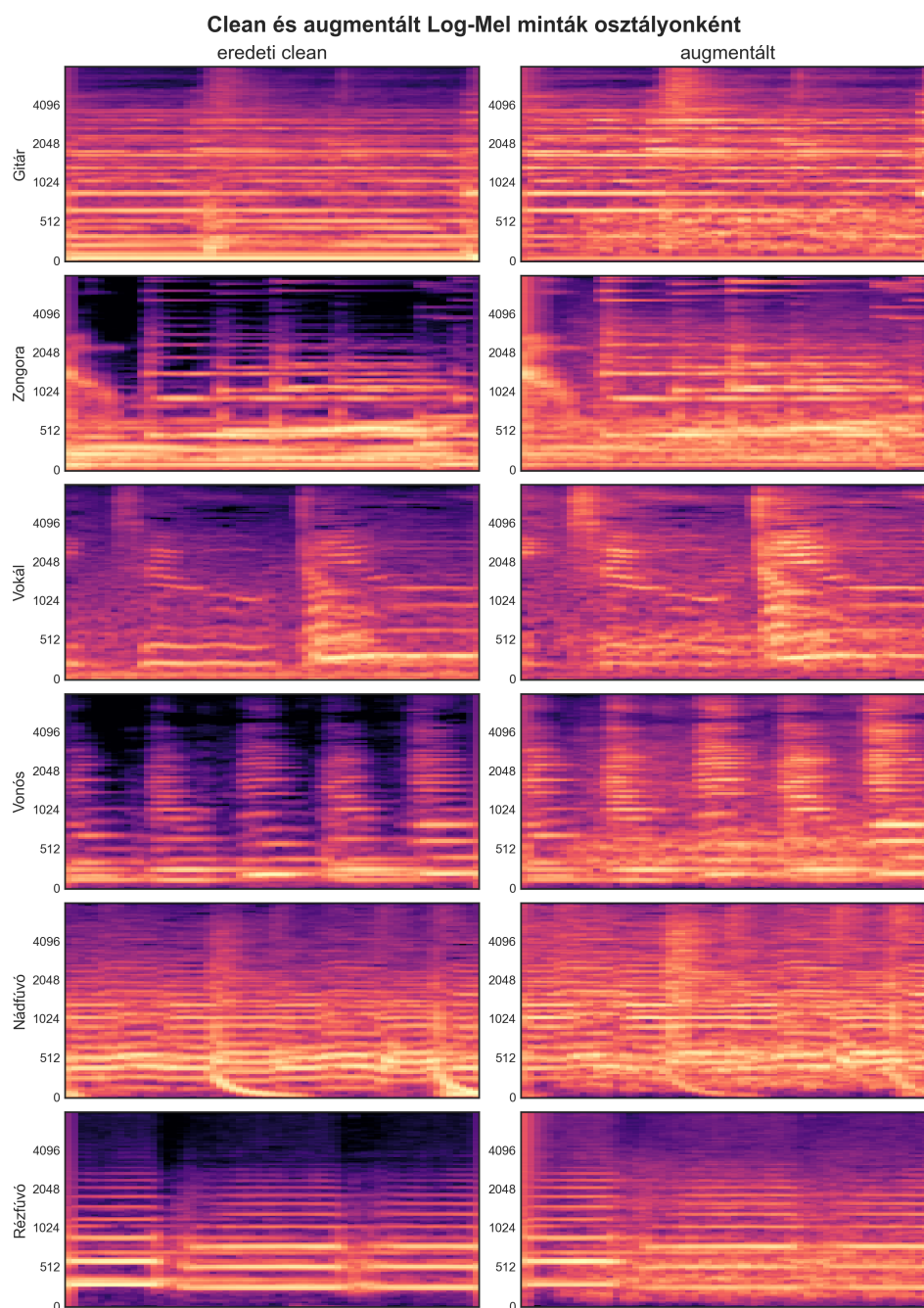
Az augmentációs lánc fő elemei:

- **szobai visszhang:** egyszerű FIR alapú reverb, véletlenszerű késleltetésekkel és csillapítással;

- **pre-emphasis:** a mikrofon és a szoba frekvenciaválaszának durva közelítése;
- **zajkeverés:** ESC-50-ből választott háttérzaj hozzáadása 5–15 dB SNR tartományban;
- **soft clipping:** enyhe nemlineáris torzítás a mikrofonbemenet telítődésének szimulálására;
- **csúcs-normalizálás:** az amplitúdó kontrollált tartományban tartása.



4.3.1. ábra: A MacBook/mikrofonos felvételi körülményeket közelítő adataugmentációs lánc



4.3.2. ábra: Reprezentatív minták normalizált Log-Mel spektrogramjai eredeti és augmentált állapotban

Az augmentáció nem a valós mikrofonos felvétel tökéletes helyettesítése. Inkább egy olcsó és jól kontrollálható regularizációs eszköz, amely segít abban, hogy a modell ne csak steril, tiszta spektrális mintázatokra tanuljon rá.

4.4. Modellarchitektúrák tervezése

A saját rendszer fő modellje egy Log-Mel spektrogramot feldolgozó 2D konvolúciós neurális hálózat. A választás oka nem az volt, hogy minden lehetséges klasszikus és ne-

urális modellváltozatot kimerítően összehasonlítsak, hanem hogy a bemenethez illeszkedő, magyarázható és valós időben is futtatható osztályozót építsek.

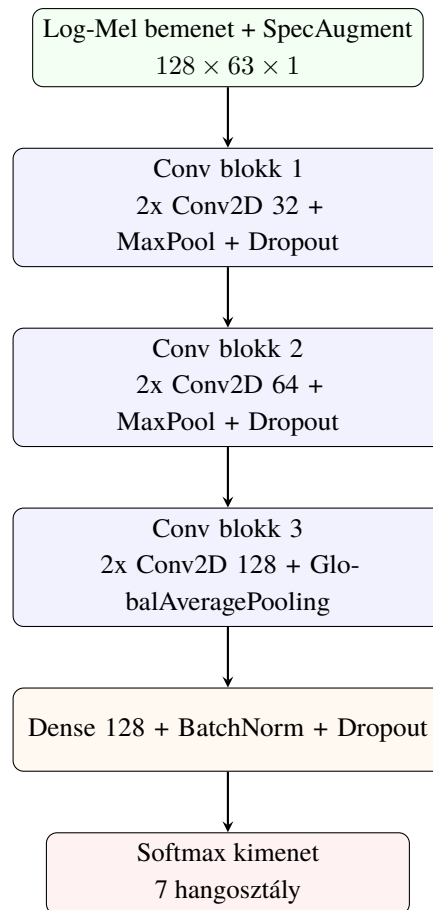
Az SVM vagy egy egyszerű MLP működhetne kisebb, aggregált jellemzővektorokon, de kevésbé használná ki a spektrogram lokális idő-frekvencia szerkezetét. Az 1D CNN inkább nyers hullámformára vagy időbeli jellemzősorozatra természetes, a CRNN pedig erősebb, de összetettebb megoldás lenne. A 2D CNN ezért ebben a dolgozatban arányos mérnöki kompromisszum: elég kifejező, de nem túl nagy.

4.4.1. A saját 2D CNN részletei

A hálózat VGG-szerű, három konvolúciós blokkot tartalmazó architektúra. A bemeneten SpecAugment réteg található, amely tanítás közben idő- és frekvenciaszűréseket alkalmaz. A konvolúciós blokkok Conv2D, Batch Normalization, ReLU, MaxPooling és Dropout elemekből állnak. A Flatten helyett GlobalAveragePooling2D réteget használok, hogy a paraméterszám kezelhető maradjon.

4.4.1. táblázat: A tervezett 2D CNN architektúra rétegei (Log-Mel bemenet: $128 \times 63 \times 1$)

Réteg	Szűrők / Neuronok	Kimenet	Paraméterek
SpecAugment	max 15 frekvencia, 8 idő maszk	$128 \times 63 \times 1$	0
$2 \times \text{Conv2D} + \text{BN} + \text{ReLU}$	32 szűrő, 3×3	$128 \times 63 \times 32$	$320 + 128 + 9\,248 + 128$
MaxPool2D + Dropout	2×2 , dropout: 0,3	$64 \times 31 \times 32$	0
$2 \times \text{Conv2D} + \text{BN} + \text{ReLU}$	64 szűrő, 3×3	$64 \times 31 \times 64$	$18\,496 + 256 + 36\,928 + 256$
MaxPool2D + Dropout	2×2 , dropout: 0,3	$32 \times 15 \times 64$	0
$2 \times \text{Conv2D} + \text{BN} + \text{ReLU}$	128 szűrő, 3×3	$32 \times 15 \times 128$	$73\,856 + 512 + 147\,584 + 512$
GlobalAvgPool2D + Dropout	dropout: 0,5	128	0
Dense + BN + ReLU	128 neuron	128	$16\,512 + 512$
Dense	7, Softmax	7	903
Összesen			$\sim 306\,150$



4.4.1. ábra: A saját Log-Mel + 2D CNN modell felépítése

4.4.2. YAMNet transfer learning referencia

A saját CNN mellett YAMNet alapú transfer learning modellt is terveztem összehasonlításra. A YAMNet egy AudioSeten előtanított, MobileNetV1 alapú audio modell, amely nyers, 16 kHz-es hullámformából 1024 dimenziós embeddinget állít elő. A rendszeremben ezt a modellt befagyasztott jellemzőkinyerőként használok, az embeddingre pedig egy kisebb, két rejtett rétegből álló Dense osztályozót tanítok.

Ez a megközelítés azért hasznos referencia, mert a nehéz akusztikai reprezentációt nem a saját, kis adathalmazon kell nulláról megtanulni. Ugyanakkor a dolgozat fő saját megoldása továbbra is a Log-Mel + 2D CNN pipeline marad, mivel ez épül be a valós idejű felismerő modulba.

4.4.3. Tanítási konfiguráció

A tanítást úgy állítottam be, hogy a kis adathalmaz mellett csökkentse a túlilleszkedés kockázatát, és az ismételt futások eredményei visszakereshetők legyenek. A legfontosabb konfigurációkat a 4.4.2. táblázat foglalja össze.

4.4.2. táblázat: A saját 2D CNN tanításához használt fő konfiguráció

Paraméter / Beállítás	Érték / Részletek
Optimalizáló	AdamW, tanulási ráta: 10^{-3} , weight decay: 10^{-4}
Veszteségfüggvény	Categorical Crossentropy, 0,1 label smoothing
Batch méret	32
Maximális epochszám	150
Early Stopping	val_loss, patience = 10, legjobb súlyok visszaállítása
ReduceLROnPlateau	val_loss, patience = 4, faktor = 0,5
ModelCheckpoint	val_loss alapján, időbélyeges .keras fájlba

A checkpoint mentés val_loss alapján történik, mert ez jobban jelzi a modell általánosítását, mint egyetlen epoch validációs accuracy értéke. A models/ mappába minden futás időbélyeges riportot, JSON állományt, konfúziós mátrixot, F1-ábrát és tanulási görbét ment. Így az ismételt validációs futások nem írják felül egymást.

4.4.3.1. A tanítás során generált kimenetek

Egy tipikus tanítás a következő kimeneteket hozza létre:

- **checkpoint:** ..._best_model.keras,
- **classification report:** szöveges és JSON formában,
- **konfúziós mátrix:** CSV és PNG formában,
- **diagramok:** osztályonkénti F1-score és tanulási görbe.

Ezek a kimenetek nemcsak a dolgozat eredményeinek bemutatására szolgálnak, hanem gyakorlati fejlesztői szempontból is fontosak: visszakereshetővé teszik, melyik modell melyik adathalmazon és melyik futásban készült.

4.5. A valós idejű felismerő modul tervezése

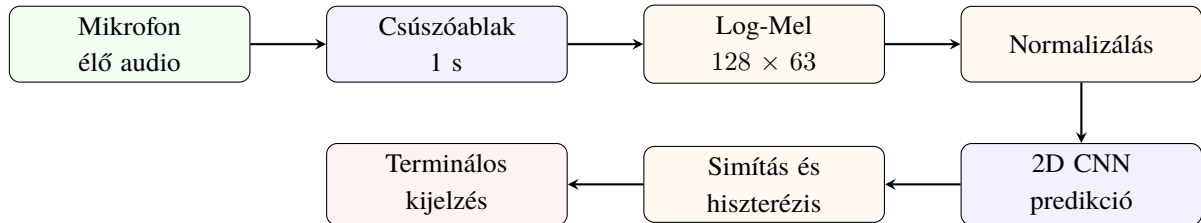
A valós idejű modul feladata, hogy a mikrofonból érkező hangot ugyanabba a Log-Mel formába alakítsa, amelyen a saját CNN tanult, majd stabilizált predikciót jelenítsen meg. A használati részletek a 6. fejezetbe tartoznak, itt csak a tervezési döntéseket foglalom össze.

4.5.1. Audio stream és csúszóablak

A mikrofonos bemenetet a sounddevice könyvtár InputStream API-ja kezeli. A rendszer 16 kHz-es, mono hangot olvas, 0,25 másodperces lépésközzel. A predikció mindig az utolsó 1 másodperces csúszóablakból készül, így a rendszer másodpercenként négyszer frissítheti a döntést, miközben a modell bemenete egyezik a tanítási minták hosszával.

4.5.2. Normalizálás és stabilizálás

Minden predikciós ciklusban 128×63 méretű Log-Mel mátrix készül. A valós idejű normalizálás a tanítási pipeline-nal kompatibilis, fix dB-tartományra épülő $[0, 1]$ skálázást használ. A pillanatnyi predikciók ingadozását egy 6 elemű mozgóátlag puffer, hiszterézis, valamint külön zaj- és hangszerküszöb csökkenti.

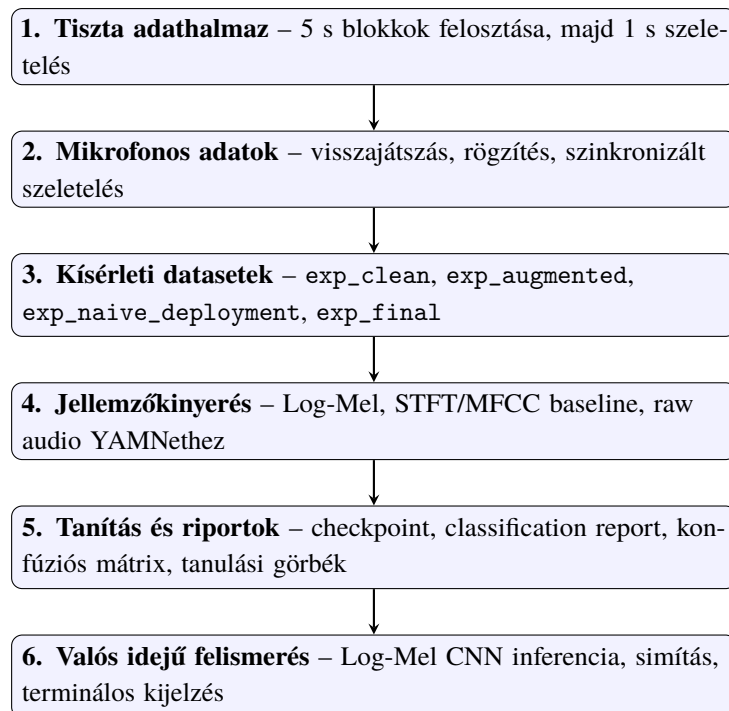


4.5.1. ábra: A valós idejű Log-Mel + 2D CNN felismerési folyamat

4.6. A feldolgozási folyamat összefoglalása

A teljes rendszer egymásra épülő, külön futtatható fázisokból áll. Ez a modularitás segített abban, hogy az adatelőkészítés, a jellemzőkinyerés, a tanítás és a valós idejű felismerés külön is ellenőrizhető legyen.

1. **Tiszta adathalmaz előkészítése:** forrásblokkok szétoztása, majd 1 másodperces szelektelés.
2. **Mikrofonos újrafelvétel:** split szerint rendezett minták visszajátszása, rögzítése és újraszelektelése.
3. **Kísérleti adathalmazok létrehozása:** clean, augmented, naive deployment és final változatok.
4. **Jellemzőkinyerés:** Log-Mel, clean baseline esetén STFT és MFCC, YAMNethez nyers hullámforma.
5. **Modelltanítás:** saját 2D CNN és YAMNet transfer learning modellek.
6. **Valós idejű felismerés:** kiválasztott saját Log-Mel CNN checkpoint betöltése és mikrofonos predikció.



4.6.1. ábra: A hangszerfelismerő rendszer végső adatfeldolgozási és tanítási folyamata

5. ÜZEMBE HELYEZÉS ÉS KÍSÉRLETI EREDMÉNYEK

Ebben a fejezetben az elkészült rendszer gyakorlati üzembe helyezését és a végső kísérleti eredményeket mutatom be. A hangsúly itt már nem azon van, hogy milyen komponensekből épül fel a rendszer, hanem azon, hogy a megtervezett feldolgozási folyamat ténylegesen hogyan viselkedik mérés közben. Igyekeztem nem egyetlen jól sikerült futásra támaszkodni, hanem több ismétlésből levonni következtetést, mert kis adathalmaz és neurális háló esetén a véletlen inicializáció és a tanítás lefutása szemmel láthatóan befolyásolja a végeredményt.

A fejezet fő mérései a saját Log-Mel spektrogram alapú 2D CNN modellre vonatkoznak, mert ez adja a dolgozat saját, valós időben használt felismerő pipeline-ját. Kiegészítésként bemutatom a YAMNet alapú transzfer tanulási összehasonlítást is, amely erős előtanított audio reprezentációra épül, de nem váltja ki a saját megoldást.

5.1. Üzembe helyezési lépések

A valós idejű felismerő alkalmazás a saját, Log-Mel bemenetű 2D CNN modellt használja. Ez fontos különbség a transzfer tanulós megközelítéssel szemben: a valós idejű demóban nem egy külső, előtanított modell adja a végső felismerést, hanem a dolgozat során felépített saját feldolgozási lánc.

A rendszer futtatásához a következő fő lépések szükségesek:

1. **Python környezet előkészítése.** A projekt futtatásához Python 3.10 vagy újabb verziót használtam, külön virtuális környezettel:

```
python3 -m venv venv
source venv/bin/activate
```

2. **Függőségek telepítése.** A szükséges csomagokat a `requirements.txt` alapján lehet telepíteni:

```
pip install -r requirements.txt
```

A rendszer főbb könyvtárai a `tensorflow`, a `librosa`, a `numpy`, a `matplotlib`, valamint a mikrofonos bemenethez használt `sounddevice`. macOS alatt a `sounddevice` használatához a `PortAudio` telepítése is szükséges:

```
brew install portaudio
```

3. **A végső modell kiválasztása.** A tanítás során minden futás külön időbélyeges checkpointot hoz létre a `models/` mappa megfelelő alkönyvtárában. A végső valós idejű futtatáshoz az `exp_final` kísérlet validáción legjobb Log-Mel modelljét használtam.

4. **Valós idejű felismerő indítása.** A saját modellre épülő valós idejű felismerő az alábbi paranccsal indítható:

```
python scripts/05a_realtime_recognition.py
```

A valós idejű alkalmazásban a bemeneti hang egy 1 másodperces csúszóablakon keresztül kerül feldolgozásra. A modell minden ablakból Log-Mel spektrogramot számol, majd a predikciókat rövid mozgóátlagos simítással stabilizálja. Ez nem javítja meg a modellt statisztikai értelemben, de a felhasználói oldalon sokat számít: a kijelzés kevésbé ugrál, és a bizonytalan pillanatnyi predikciók nem jelennek meg azonnal végleges döntésként.

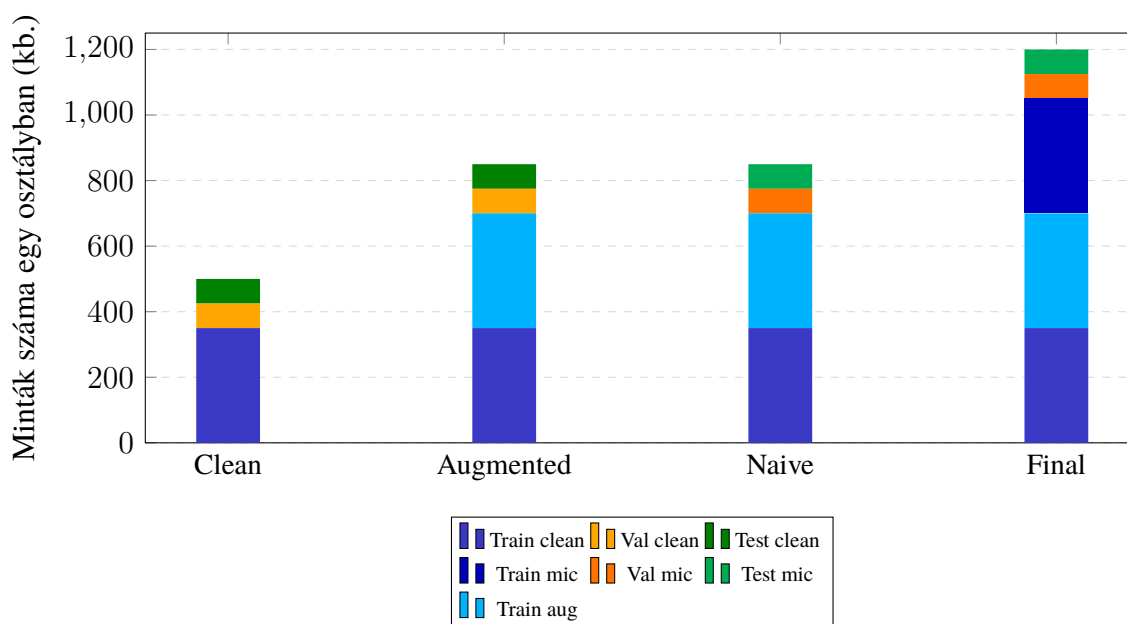
5.2. Kísérleti protokoll

A mérések célja nem csak az volt, hogy kapjak egy végső pontossági értéket, hanem az is, hogy külön-külön látszódjon az augmentáció, a domain shift és a mikrofonos célkörnyezeti adatok hatása. Emiatt négy kísérleti adathalmazt állítottam össze. Ezek közelítő méretét a 5.2.1. táblázat foglalja össze. A táblázatban az osztályonkénti ideális, kerekített mennyiségeket használom, mert a kísérleti felépítés így sokkal könnyebben átlátható.

5.2.1. táblázat: A négy kísérleti adathalmaz közelítő, osztályonként levezetett mérete

Kísérlet	Train	Val	Test
Clean baseline	7×350 ≈ 2450	clean $7 \times 75 \approx 525$	clean $7 \times 75 \approx 525$
Augmented	$7 \times (350 + 350)$ ≈ 4900	clean $7 \times 75 \approx 525$	clean $7 \times 75 \approx 525$
Naive deployment	$7 \times (350 + 350)$ ≈ 4900	mic $7 \times 75 \approx 525$	mic $7 \times 75 \approx 525$
Final system	$7 \times (350 + 350 + 350)$ ≈ 7350	mic $7 \times 75 \approx 525$	mic $7 \times 75 \approx 525$

A táblázatban és a 5.2.1. ábrán szándékosan kerekített értékeket használok. A kísérleti fázisok logikája így áttekinthetőbb: nem az egyes osztályok néhány mintás eltérése a lényeg, hanem az, hogy egy osztályon belül az eredeti 70% – 15% – 15%-os felosztás körülbelül 350-75-75 mintát jelent. A clean baseline a tiszta internetes adatokat adja, az augmented kísérlet a szoftveres augmentáció hatását méri, a naive deployment a mikrofonos validációs/tesztkörnyezetre való átvitel nehézségét mutatja, a final system pedig már a mikrofonos tanítóanyaggal kiegészített végső felállítás. Az augmentált és mikrofonos tanítóanyag tehát a train részt növeli, miközben a validációs és tesztadatok külön maradnak. Az ábrán a train, validation és test részek külön színcsaládot kaptak, ezeken belül pedig az árnyalat jelzi, hogy clean, augmentált vagy mikrofonos adatról van szó.



5.2.1. ábra: A kísérleti adathalmazok közelítő megoszlása egy átlagos osztályra vetítve

A modellválasztást minden kísérletnél a validációs halmazon végeztem. Minden kísérletet ötször futtattam le egymás után, majd az eredményeket átlag és szórás formájában összesítettem. Erre azért volt szükség, mert a kis méretű adathalmaz és a neurális háló sztochasztikus tanítása miatt egyetlen futás eredménye nem adna elég stabil képet. A végső teszhalmazt csak a modellválasztás után, egyszer használtam: az `exp_final` kísérlet legjobb validációs Macro F1 értékű checkpointját töltöttem vissza, és azt futtattam le a test halmazon.

A kísérletek során két fő mérőszámot használtam:

- **Accuracy:** az összes mintára vetített találati arány.
- **Macro F1:** az osztályonként számolt F1-score-ok egyszerű átlaga, amely jobban megmutatja, ha a modell bizonyos osztályokat elhanyagol.

Hangszerfelismerésnél a Macro F1 különösen fontos, mert nem elég az, hogy a modell összességében sok mintát eltaláljon. Az is lényeges, hogy a gitár, zongora, vokál, vonós, nád- és rézfúvós, valamint zaj osztályok között ne legyen túl nagy teljesítménybeli különbség.

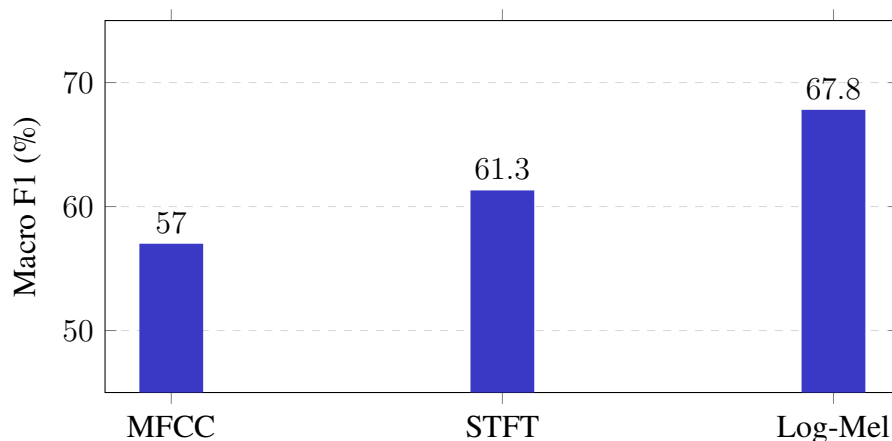
5.3. A bemeneti reprezentáció kiválasztása a clean baseline alapján

Mielőtt a négy kísérleti fázist végigfuttattam volna, külön meg kellett indokolni, hogy a saját CNN modell milyen bemeneti reprezentációval dolgozzon tovább. A rendszer tervezési részében három jellegzetes jellemzőkinyerési módszert vizsgáltam: MFCC-t, STFT spektrogramot és Log-Mel spektrogramot. Ezek közül nem elméleti alapon választottam ki egyet, hanem a clean baseline adathalmazon mindhárom változatot ötször lefuttattam ugyanazzal a 2D CNN architektúrával.

Ez a mérés azért került a tiszta baseline-ra, mert itt még nincs jelen az augmentáció vagy a mikrofonos domain shift hatása. Így a különbség elsősorban azt mutatja meg, hogy az adott reprezentáció mennyire használható a saját modell számára ideálisabb, tisztább bemenet mellett. A tesztalmaidat ebben a lépésben sem használtam: a választás kizárólag validációs eredmények alapján történt.

5.3.1. táblázat: Jellemzőkinyerési módszerek összehasonlítása a clean baseline validációs futásain

Bemeneti reprezentáció	Accuracy	Macro F1
MFCC	$56,7 \pm 4,6\%$	$57,0 \pm 5,0\%$
STFT	$61,8 \pm 5,9\%$	$61,3 \pm 6,1\%$
Log-Mel	$67,7 \pm 6,2\%$	$67,8 \pm 6,3\%$



5.3.1. ábra: A clean baseline validációs Macro F1 értékei a három bemeneti reprezentáció esetén

A 5.3.1. táblázat és a 5.3.1. ábra alapján a Log-Mel spektrogram adta a legerősebb kiindulópontot. Az MFCC a leggyengébb eredményt hozta, ami érthető: bár tömör és klasszikusan jól használható reprezentáció, a DCT lépés miatt olyan finom spektrális részleteket is elveszíthet, amelyek egy CNN számára hasznosak lennének. Az STFT ennél jobb volt, mert több nyers frekvenciainformációt őriz meg, viszont a lineáris frekvenciatengely miatt nagyobb és kevésbé hallásközelit bemenetet ad.

A Log-Mel ezzel szemben jó kompromisszumnak bizonyult. Megőrzi a képszerű idő-frekvencia szerkezetet, amelyet a 2D CNN ki tud használni, miközben a Mel-skála az emberi hallás szempontjából fontosabb tartományokat hangsúlyosabban kezeli. Emiatt a további kísérleti fázisokat – augmentáció, naive deployment és final system – már Log-Mel bemenettel futtattam. Ezzel a döntéssel a későbbi összehasonlításokban nem egyszerre változik a reprezentáció és az adathalmaz, hanem a bemenet rögzített marad, és tisztábban láthatóvá válik az adatok összetételének hatása.

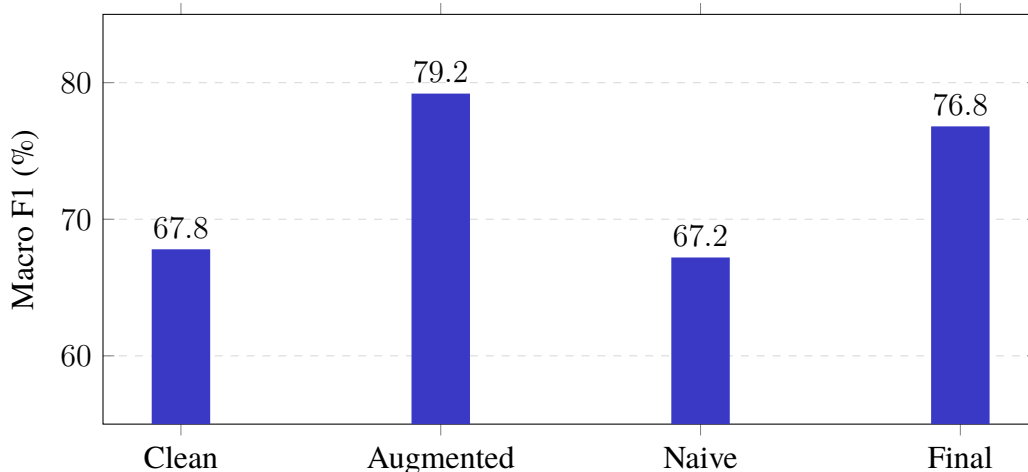
5.4. Validációs eredmények

Az öt-öt validációs futás összesítését a 5.4.1. táblázat mutatja. Innentől kezdve minden saját CNN kísérleti fázis a kiválasztott Log-Mel reprezentációval futott. A számok alapján jól látszik, hogy a különböző kísérleti fázisok nem csak abszolút pontosságban, hanem stabilitásban is eltérnek egymástól.

5.4.1. táblázat: Validációs eredmények öt futás alapján

Kísérlet	Accuracy	Macro F1
Clean baseline	$67,7 \pm 6,2\%$	$67,8 \pm 6,3\%$
Augmented	$79,2 \pm 2,2\%$	$79,2 \pm 2,2\%$
Naive deployment	$67,3 \pm 3,6\%$	$67,2 \pm 3,9\%$
Final system	$76,9 \pm 1,8\%$	$76,8 \pm 2,1\%$

A 5.4.1. ábra ugyanezt a folyamatot vizuálisan mutatja. Az ábra nem egyetlen futás szerencsáját ábrázolja, hanem az öt futás átlagos Macro F1 értékét.



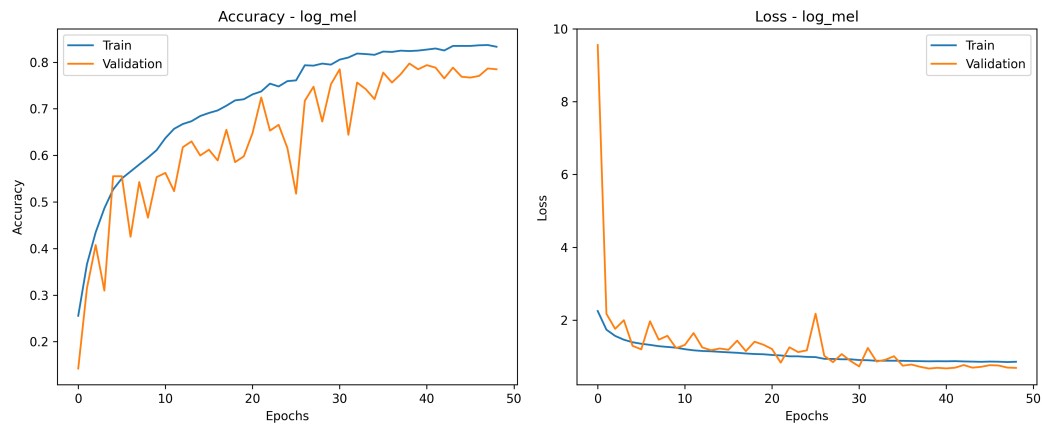
5.4.1. ábra: A validációs Macro F1 értékek alakulása a négy kísérleti fázisban

Az eredmények alapján az augmentáció hatása egyértelműen pozitív. A tiszta baseline 67,8%-os Macro F1 értékéhez képest az augmentált kísérlet 79,2%-ot ért el. Ez azt jelenti, hogy a mesterségesen hozzáadott torzítások, zajok és szobaszerű visszaverődések nem csak több adatot adtak a modellnek, hanem valóban segítettek a robusztusabb mintázatok megtanulását.

A naive deployment kísérlet ezzel szemben visszaesett 67,2%-os Macro F1 értékre. Ez a mérés számomra a dolgozat egyik legfontosabb tanulsága: ha a modellt tiszta és szoftveresen augmentált adatokon tanítjuk, majd mikrofonos környezetben próbáljuk használni, a domain shift mérhető teljesítményromlást okoz. Más szóval a valós idejű működéshez nem elég a letöltött vagy tisztított hanganyag, mert a konkrét mikrofon, hangszóró, szoba és zajprofil is beleszól abba, mit lát a modell.

A final system kísérletben a mikrofonos train adatok is bekerültek a tanításba. Ennek hatására a Macro F1 76,8%-ra emelkedett, vagyis a rendszer a naiv mikrofonos kiértékeléshez

képest a teljesítmény nagy részét visszanyerte. Ez nem azt jelenti, hogy a modell hibátlan lett, de mérnöki szempontból jól látható, hogy a célkörnyezeti adatok bevonása működő domain adaptation lépésnek bizonyult.



5.4.2. ábra: A legjobb validációs *exp_final* futás tanulási görbéi

A 5.4.2. ábrán látható tanulási görbék alapján a modell tanítása nem teljesen sima folyamat. Ez várható is: a validációs halmaz néhány száz mintából áll, a bemenetek pedig valós hangmintákból és mikrofonos újrafelvételekből származnak. A görbék emiatt helyenként ugrálnak, viszont a tendencia mégis értelmezhető: a tanítás előrehaladtával a validációs teljesítmény javul, majd egy pont után már nem hoz stabil további nyereséget. Emiatt használtam early stoppingot és a validációs veszteség alapján mentett checkpointot.

5.5. Végző teszteredmények

A végző tesztet már nem használtam modellválasztásra. Először a saját Log-Mel alapú 2D CNN-t értékeltem ki. Az öt *exp_final* validációs futás közül a legjobb Macro F1 értékű checkpointot választottam ki:

```
exp_final_log_mel_2dcnn_val_20260527_222650_best_model.keras
```

Ezt a modellt újratanítás nélkül töltöttem vissza, és egyszer futtattam le a félretett teszthalmazon. A végző eredmény:

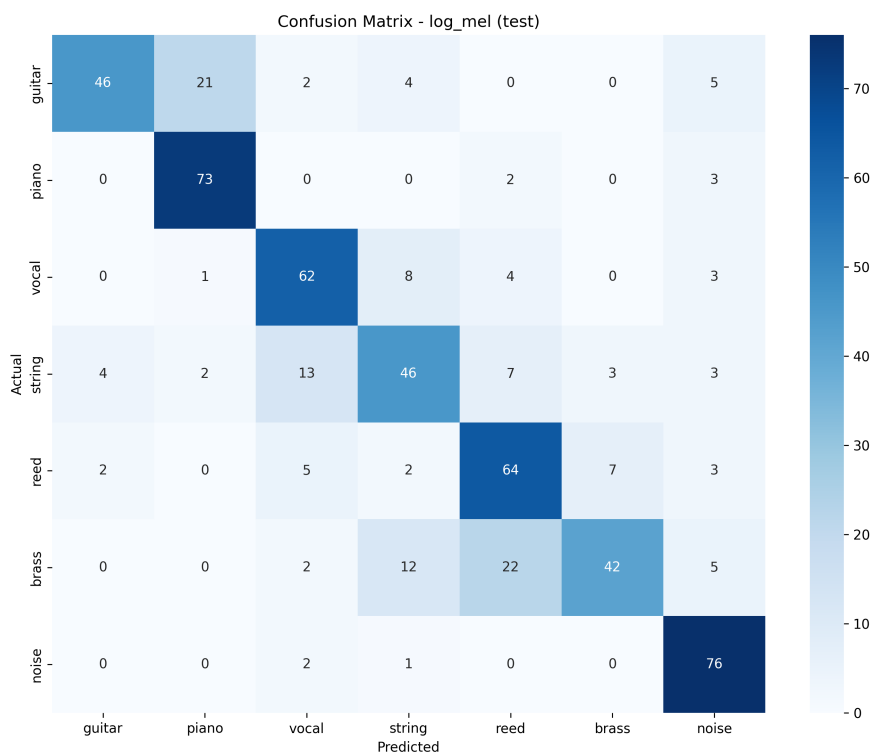
Accuracy: 73,4%, Macro F1: 72,7%

A részletes osztályonkénti eredményeket a 5.5.1. táblázat mutatja.

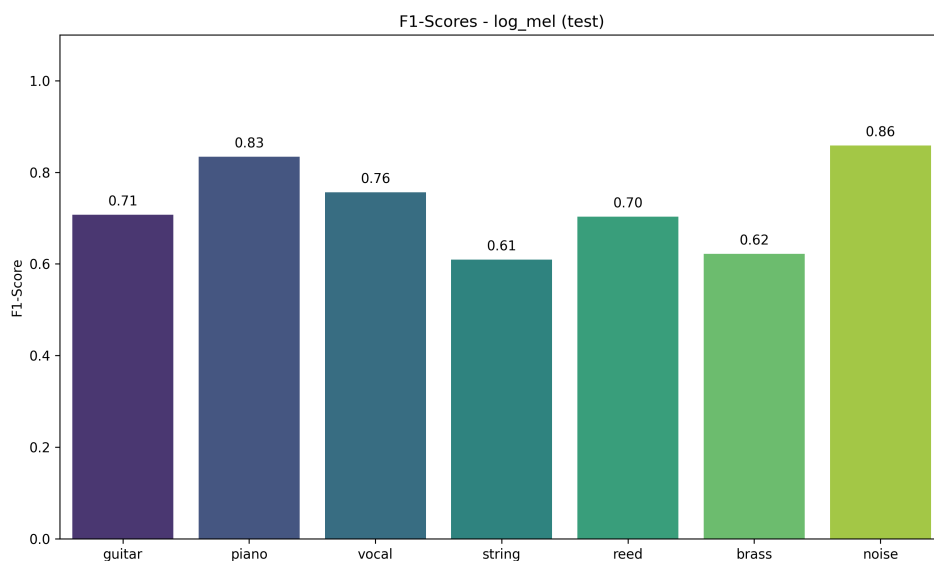
5.5.1. táblázat: A végső *exp_final* modell teszteredménye osztályonként

Osztály	Precision	Recall	F1-score	Mintaszám
Gitár	88,5%	59,0%	70,8%	78
Zongora	75,3%	93,6%	83,4%	78
Vokál	72,1%	79,5%	75,6%	78
Vonós	63,0%	59,0%	60,9%	78
Nádfúvós	64,6%	77,1%	70,3%	83
Rézfúvós	80,8%	50,6%	62,2%	83
Zaj	77,6%	96,2%	85,9%	79
Összesen / átlag	75,0%	73,4%	72,7%	557

A teszteredmény realisabb képet ad a rendszer generalizációjáról, mint a legjobb validációs futás. A validáción elért 79,7%-os Macro F1 a modellválasztáshoz hasznos volt, de a félretett tesztalmazon már 72,7%-os Macro F1 jelent meg. Ezt nem tekintem hibának, inkább fontos figyelmeztetésnek: kis saját adathalmaznál a validációs eredmény optimistább lehet, ezért a végső testmérés nélkül könnyű lenne túlértékelni a rendszert.



5.5.1. ábra: A végső *exp_final* modell konfúziós mátrixa a tesztalmazon



5.5.2. ábra: A végső *exp_final* modell osztályonkénti F1-score értékei

A 5.5.1. ábra alapján a legerősebb osztály a zaj, amely 85,9%-os F1-score-t ért el. Ez gyakorlati szempontból kedvező, mert a valós idejű alkalmazásnak nemcsak hangszert kell felismernie, hanem azt is kezelnie kell, amikor nincs értelmezhető hangszeres bemenet. A zongora szintén erős eredményt adott 83,4%-os F1-score-al, ami valószínűleg a jellegzetes tranzienseknek és a jól elkülönülő spektrális mintázatnak köszönhető.

A gyengébb pontok főleg a vonós és rézfúvós osztályoknál jelentkeztek. A rézfúvós osztály recall értéke csak 50,6%, vagyis a valódi rézfúvós minták közel fele más osztályba került. A konfúziós mátrix szerint ezek jelentős része nádfúvós vagy vonós osztályként jelent meg. Ez akusztikailag nem teljesen meglepő: egyes rézfúvós és nádfúvós hangok felhangszerkezete, megszólalási karaktere és a mikrofonos torzítás miatt látható Log-Mel mintázata közel kerülhet egymáshoz.

A gitár esetében magas precision, de alacsonyabb recall figyelhető meg. Ez azt jelenti, hogy amikor a modell gitárt mond, akkor többnyire igaza van, viszont sok valódi gitármintát nem mer gitárként azonosítani. A hibák egy része zongora irányba megy, ami a pengetett és leütött húrok kezdeti tranziensének hasonlósága miatt értelmezhető.

5.5.1. YAMNet transfer learning összehasonlítás

A saját modell mellett lefuttattam a YAMNet alapú transzfer tanulási változatot is. Ennél a YAMNet befagyasztott előtanított hálóként működik: a nyers 1 másodperces audiojelből 1024 dimenziós embeddinget készít, és csak az erre épített kisebb osztályozófejet tanítom a saját hangszerosztályokra. A validációs futás 83,6%-os accuracy-t és 83,7%-os Macro F1 értéket adott, a kiválasztott checkpoint egyszeri tesztfuttatása pedig a következő eredményt hozta:

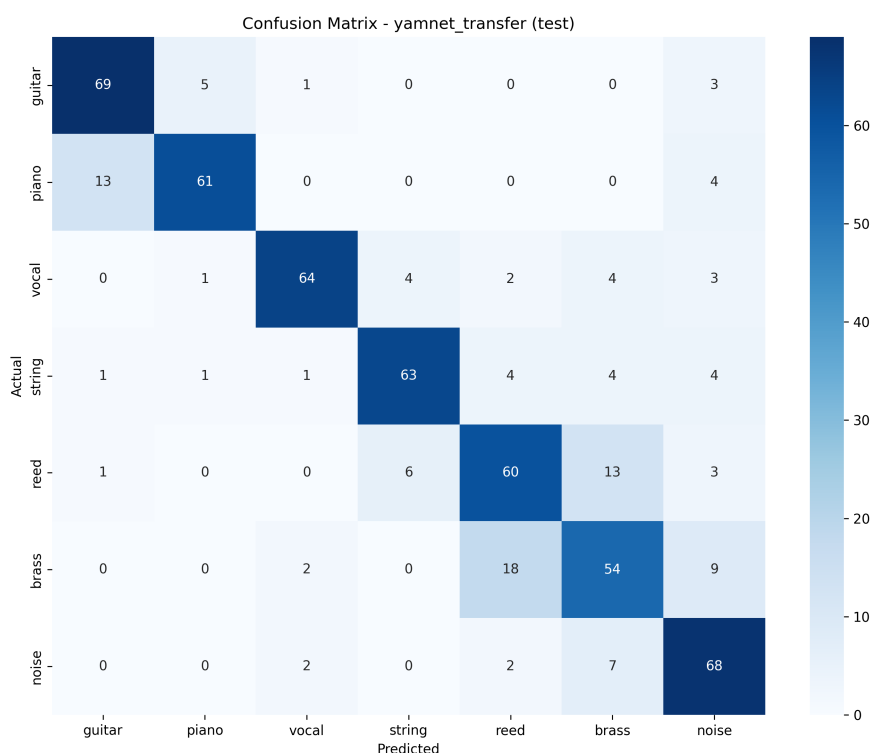
YAMNet transfer learning – Accuracy: 78,8%, Macro F1: 79,1%

5.5.2. táblázat: A saját Log-Mel modell és a YAMNet transfer learning teszteredményeinek összehasonlítása

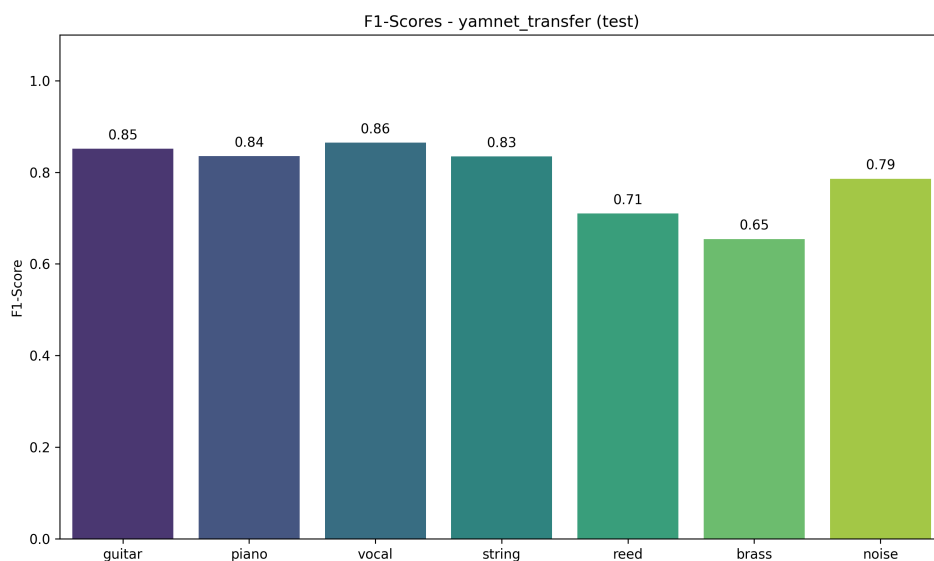
Modell	Accuracy	Macro F1
Saját Log-Mel + 2D CNN	73,4%	72,7%
YAMNet transfer learning	78,8%	79,1%

A YAMNet tehát nem csak minimálisan, hanem mérhetően jobb eredményt adott: körülbelül 5,4 százalékponttal magasabb accuracy-t és 6,4 százalékponttal magasabb Macro F1 értéket ért el. Ugyanakkor ez nem nagyságrendi különbség. Ennek több oka is van: a teszt-halmaz mikrofons domainből származik, a hangszerosztályok több konkrét hangszert vonnak össze egy-egy kategóriába, az 1 másodperces minták sokszor csak rövid részletet tartalmaznak, a YAMNet pedig általános AudioSet-feladatokra előtanított modell, nem kifejezetten erre a hét osztályra készült.

Osztályszinten a YAMNet főleg a gitár, vokál és vonós kategóriákon javított sokat. A zaj osztálynál viszont a saját Log-Mel modell maradt erősebb, ami arra utal, hogy a saját pipeline a konkrét zajkülönbölési és mikrofons környezethez bizonyos szempontból jobban illeszkedett.



5.5.3. ábra: A YAMNet transfer learning modell konfúziós mátrixa a teszt-halmazon



5.5.4. ábra: A YAMNet transfer learning modell osztályonkénti F1-score értékei

5.6. A jelenlegi rendszer értékelése

A mérési eredmények alapján a projekt jelenlegi állapotában nem rekordpontosságú hangszerfelismerő rendszer, hanem egy végigvezetett, működő és mérhető mérnöki megoldás. A rendszer erőssége nem abban van, hogy minden osztályban közel tökéletes eredményt ad, hanem abban, hogy a teljes út reprodukálható: az adatgyűjtéstől és szelekteléstől kezdve az augmentáción, mikrofonos újrafelvételen, jellemzőkinyerésen és tanításon át a valós idejű predikcióig.

A legfontosabb következtetések:

- Az augmentáció a validációs eredmények alapján erős javító hatást mutatott.
- A mikrofonos környezetre való közvetlen átvitel érezhető domain shiftet okozott.
- A mikrofonos train adatok bevonása a célkörnyezeti teljesítményt jelentősen javította.
- A végső teszteredmény szerényebb, mint a legjobb validációs futás, de realisabb képet ad a rendszer tényleges generalizációjáról.
- A YAMNet transfer learning erős összehasonlítási pontot adott, de a saját Log-Mel pipeline továbbra is a dolgozat fő, valós időben használt megoldása maradt.

Módszertani szempontból fontos korlát, hogy az adathalmaz mérete továbbra is viszonylag kicsi, és a saját CNN tanításai futásonként ingadoznak. Emiatt a köztes kísérletek között nem minden különbséget érdemes túl erősen értelmezni. Az viszont stabilan látszik, hogy az augmentáció és a célkörnyezeti mikrofonos adatok használata javító irányba mozdította el a rendszert. A YAMNet eredménye azt is megmutatta, hogy egy nagy, előtanított audio reprezentációval lehet még javítani a pontosságon, de a saját pipeline önmagában is működő, mérhető és valós időben használható megoldás maradt. Ez a dolgozat céljához illeszkedik:

nem egy lezárt, ipari pontosságú terméket készítettem, hanem egy saját adatokon végigvezetett hangszerfelismerő rendszert, amelynek előnyei és korlátai is mérhetően megjelennek.

6. A RENDSZER FELHASZNÁLÁSA

Az előző fejezetben bemutattam, hogyan helyeztem üzembe a rendszert, milyen kísérleti protokollt használtam, és milyen eredményeket kaptam a végső Log-Mel alapú 2D CNN modellel. Ebben a fejezetben már nem a tanítási és mérési oldalt ismételtem meg, hanem azt mutatom be, hogyan használható a rendszer felhasználói szemszögből: mit lát a program indításakor az ember, hogyan működik a valós idejű konzolos visszajelzés, és milyen helyzetekben viselkedik stabilan vagy bizonytalanabban.

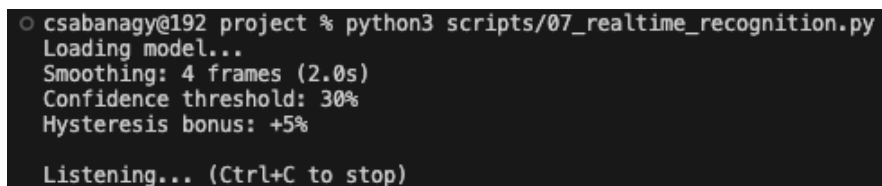
A rendszer jelenlegi formájában nem grafikus alkalmazás, hanem parancssori felismerő eszköz. Ezt tudatos döntésként kezeltem: a célom nem egy látványos felület elkészítése volt, hanem egy gyorsan futtatható, kevés erőforrást igénylő, valós időben reagáló prototípus. Emiatt a felület egyszerű, de a fontos információkat közvetlenül mutatja: az aktuális osztályt, a hozzá tartozó konfidenciát és a többi osztály pillanatnyi valószínűségét.

6.1. Az alkalmazás indítása

A telepítési és futtatási előfeltételeket a 5.1. szakaszban már összefoglaltam. A használat szempontjából a lényeg az, hogy a virtuális környezet aktiválása és a mikrofon kiválasztása után a valós idejű felismerő modul az alábbi paranccsal indítható:

```
python scripts/05a_realtime_recognition.py
```

Indításkor a program betölti a kiválasztott Keras modellt, majd megjeleníti a futás közbeni legfontosabb beállításokat. A jelenlegi konfigurációban a scriptben a `MODEL_TYPE = "exp_final"` beállítás aktív, ezért alapértelmezetten az `exp_final` modellmappából dolgozik. Ha a `CHECKPOINT_NAME` változó nincs kézzel beállítva, akkor a program automatikusan a legfrissebb Log-Mel alapú checkpointot választja ki a `models/exp_final/` könyvtárból. Ez kényelmes a mindennapi futtatásnál, de reprodukálható bemutatóhoz természetesen kézzel is megadható egy konkrét checkpointnév.



```
csabanagy@192 project % python3 scripts/07_realtime_recognition.py
Loading model...
Smoothing: 4 frames (2.0s)
Confidence threshold: 30%
Hysteresis bonus: +5%

Listening... (Ctrl+C to stop)
```

6.1.1. ábra: Konzolos példa az alkalmazás indításakor megjelenő konfigurációs adatokra

A 6.1.1. ábra egy tipikus indítási állapotot szemléltet. Itt nem külön mérési eredményről van szó, hanem arról, hogy a program visszajelzi: melyik modellt használja, mekkora simítási ablakot alkalmaz, és milyen konfidenciaküszöbök mellett dönt az aktív osztály megjelenítéséről.

2. **Aktív osztály:** az aktuálisan megjelenített hangszer vagy zaj kategória neve, százalékos konfidenciával.
3. **Másodlagos valószínűségek:** a többi osztály rövidített jelölése és pillanatnyi valószínűsége, ami segít észrevenni, ha a modell két osztály között bizonytalan.

A program a terminál egyetlen sorát írja felül folyamatosan, ezért futás közben nem keletkezik hosszú, nehezen olvasható napló. Ez egyszerű megoldás, de élő bemutatónál praktikus: nem kell grafikus ablak, nincs külön vizualizációs szerver, és a program egy átlagos terminálban is használható.

6.3.1. Színkódolás és jelölések

A könnyebb olvashatóság miatt minden osztály saját színt és rövidítést kapott. Ezeket a 6.3.1. táblázat foglalja össze. A színek nem befolyásolják a predikciót, csak a gyorsabb emberi értelmezést segítik.

6.3.1. táblázat: A parancssori felületen alkalmazott osztályjelölések

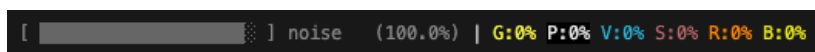
Kategória	Terminál színe	Rövidítés	Leírás
guitar	Sárga	G	Gitár jellegű pengetős hangok
piano	Inverz fekete/fehér	P	Zongora és billentyűs jellegű hangok
vocal	Ciánkék	V	Emberi énekhang
string	Sötétvörös	S	Vonós hangszerek
reed	Narancssárga	R	Nádas fúvós hangszerek
brass	Világosbarna	B	Rézfúvós hangszerek
noise	Szürke	N	Környezeti zaj vagy csend

6.4. Működési forgatókönyvek

Az alábbi példák azt mutatják meg, hogyan jelenik meg a rendszer kimenete tipikus használati helyzetekben. Ezek nem külön statisztikai mérések, hanem konzolos működési példák. A számszerű modellértékelést az előző fejezet tartalmazza.

6.4.1. Csendes vagy zajos szobai környezet

Alaphelyzetben, amikor nincs aktív hangszeres játék, a rendszernek nem szabad minden apró zörejt hangszerként kezelnie. A végső teszten a zaj osztály bizonyult az egyik legerősebb kategóriának, 85,9%-os F1-score értékkel. Ez a használat során is fontos, mert egy valós idejű felismerő alkalmazás gyakran többet hall csendből, szobazajból és háttérzörejből, mint tényleges hangszerből.



6.4.1. ábra: Konzolos példa zaj vagy csend felismerésekor

A 6.4.1. ábrán látható állapotban a noise osztály jelenik meg aktívként. Ilyenkor a felület nem azt állítja, hogy a környezet tökéletesen néma, hanem azt, hogy a modell számára nincs olyan domináns hangszeres mintázat, amely valamelyik hangszerosztályhoz erősebben kötődne.

6.4.2. Gitárjáték

Gitár esetén a modell viselkedése óvatosabban értelmezendő. A végső teszten a gitár precision értéke magas volt (88,5%), vagyis amikor a modell gitárt jelzett, általában jó döntést hozott. A recall viszont csak 59,0% lett, ami azt mutatja, hogy nem minden valódi gitármintát talált meg gitárként. Ez használat közben úgy jelenhet meg, hogy a gitár stabil felismerése erősen függ a hangszíntől, a játékmódtól és a mikrofonba érkező jel tisztaságától.

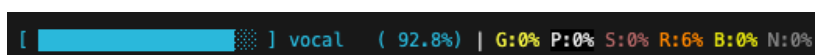


6.4.2. ábra: Konzolos példa gitárként azonosított bemenet esetén

A 6.4.2. ábra ezért nem általános garancia minden gitárhangra, hanem egy olyan pillanatot mutat, amikor a modell a bemenetet gitárként értelmezte. Ez a különbség fontos: a rendszer használható demóként, de a mérési eredmények alapján nem érdemes úgy kezelni, mint minden gitártípust egyformán biztosan felismerő megoldást.

6.4.3. Emberi énekhang

A vokál osztály a végső teszten kiegyensúlyozottabb eredményt adott, 75,6%-os F1-score-al. Használat közben ez azért érdekes, mert az emberi hang sokszor nagyon eltérő lehet: beszédszerű, énekelt, halk, erősen formánsos vagy levegős. A modell ennek ellenére több esetben stabilan tudja vokálként értelmezni a bemenetet, főleg akkor, ha a mikrofonba érkező jel tiszta és a háttérzaj nem túl erős.



6.4.3. ábra: Konzolos példa vokálként azonosított bemenet esetén

A 6.4.3. ábrán látható visszajelzésnél a másodlagos osztályok értékei is hasznosak: ha például a zaj vagy más hangszerosztály valószínűsége megemelkedik, abból látszik, hogy a modell nem teljesen egyértelmű bemenetet kap.

6.4.4. Nádfúvós és rézfúvós bizonytalanság

A fúvós hangszerek jó példát adnak arra, hogy a rendszer nem csak találatokat, hanem bizonytalanságot is mutat. A végső konfúziós mátrix alapján a rézfúvós osztály több esetben nádfúvósként vagy vonósként jelent meg. Ez a használat során is előfordulhat, különösen akkor, ha a bemenet szaxofonhoz, trombitához vagy más erős felhangszerkezetű fúvós hanghoz hasonlít.



6.4.4. ábra: Konzolos példa nádfúvós jellegű bemenet esetén

A 6.4.4. ábrán látható reed kimenet ezért nemcsak egy sikeres felismerési példaként érdekes, hanem azt is szemlélteti, miért volt szükség a simításra és a hiszterézisre. Ezek nélkül a kijelzés könnyebben váltogathatna a fúvós osztályok között, ami felhasználói oldalról zavaróbb lenne, még akkor is, ha a nyers valószínűségek szakmailag értelmezhetők.

6.5. Használati korlátok

A rendszer jelenlegi állapotában működő valós idejű prototípus, de nem általános célú, minden zenei helyzetben megbízható hangszerfelismerő alkalmazás. A legfontosabb korlátok a használat során:

- **Monofón vagy domináns hangszeres bemenetre készült.** Ha egyszerre több hangszer szól, a softmax kimenet miatt a modell akkor is egyetlen fő osztályt választ.
- **A mikrofon és a környezet hatása erős.** A final rendszer már tartalmaz mikrofonos tanítóadatokat, de másik szobában, másik mikrofonnal vagy erős háttérzajban a viselkedés változhat.
- **A hasonló hangszínű osztályok között marad bizonytalanság.** Különösen a fúvós, vonós és egyes gitár/zongora jellegű tranziens hangok okozhatnak tévesztést.
- **A kijelzés simított.** A felhasználói élmény stabilabb, de emiatt a rendszer nem a pillanatnyi nyers predikciót, hanem annak rövid időablakon átlagolt változatát mutatja.

Ezeket a korlátokat nem hibaként, hanem a jelenlegi prototípus természetes határaiként kezelem. A rendszer célja az volt, hogy egy saját adatokon tanított, valós időben futtatható hangszerfelismerő folyamatot hozzak létre, amelynek működése és hibái is megfigyelhetők. Ezt a célt a használati tesztek és az előző fejezetben bemutatott mérések alapján sikerült teljesíteni.

6.6. Az alkalmazás leállítása

Az élő felismerés a terminálban a Ctrl+C billentyűkombinációval szakítható meg. Ilyenkor a program lezárja a mikrofonbemenetet, felszabadítja az audio streamet, és visszatér a parancssorba. Ez egyszerű, de a fejlesztés és bemutatás során praktikus megoldásnak bizonyult: a rendszer gyorsan indítható, gyorsan leállítható, és nem hagy maga után külön bezárandó ablakokat vagy grafikus folyamatokat.

7. KÖVETKEZTETÉSEK

A dolgozat végére érve a céloom nem egy rekordpontosságú, minden helyzetben hibátlan hangszerfelismerő rendszer bemutatása volt, hanem egy végigvezetett, mérhető és valós időben is kipróbálható mérnöki prototípus elkészítése. Ebben a fejezetben ezért röviden összefoglalom, mit valósítottam meg, hogyan viszonyul a rendszer a hasonló megközelítésekhez, és merre lehetne továbbvinni a munkát.

7.1. Megvalósítások

Megtervezni és megközelíteni egy, a cél szempontjából tökéletes, alacsony mintaszámú zenei adathalmazt szinte lehetetlen. Azonban egy igazán jót is kihívás, nem beszélve az osztályozó modell és a jellemzőkinyerő megválasztásáról és integrálásáról. Számomra igazi kihívást jelentett a rendszer élesítése. Már csak a minták kiválogatása alatt rengeteg szép hangszerez játékmal találtam szemben magam, ami csak emelte a munka iránti ambíciómat. Bár egy ilyen projekt sosincs kész, mindig van, amit javítani rajta, az eredmény minden kisebb-nagyobb hibája ellenére elégedettséggel tölt el, és büszkén foglal helyet a portfóliómban.

A munka során elért legfontosabb elméleti és gyakorlati megvalósítások az alábbi pontokban foglalhatók össze:

- Egy olyan hibrid adathalmaz létrehozása és pontos struktúrájának kialakítása, amely egyszerű módon, bárki által újra felhasználható és bővíthető más hangszerfelismerési kutatásokban is.
- Egy moduláris feldolgozási csővezeték kiépítése, amely lehetővé teszi a különböző jellemzőkinyerési eljárások (STFT, Log-Mel, MFCC) és modellek könnyű cseréjét a rendszerben.
- Az egyszerű, offline (fájl alapú) osztályozáson túl egy olyan valós idejű hangszerfelismerő alkalmazás megvalósítása, amely közvetlen mikrofon-bemenetet használ.
- A valós idejű predikciót simító és stabilizáló logika implementálása (mozgóátlag-számítás), amely megbízhatóbbá és gördülékenyebbé teszi a kijelzést.
- A 2D CNN modell optimalizálása a gyorsabb és alacsonyabb késleltetésű futás elérése érdekében, biztosítva a zökkenőmentes kiértékelést egy átlagos asztali számítógépen.

7.2. Hasonló rendszerekkel való összehasonlítás

A 2.1. szakaszban bemutatott kapcsolódó munkákhoz képest a saját rendszerem nem a legnagyobb lehetséges adathalmazt vagy a legerősebb neurális architektúrát célozta meg. A

szakirodalomban gyakoriak a nagyobb, polifón zenei anyagokra épülő rendszerek, illetve az olyan előtanított vagy mélyebb modellek, amelyek erős általánosító képességet adhatnak, de a fejlesztési és futtatási környezetük is összetettebb. Ezzel szemben én egy kisebb, átláthatóbb, saját adatgyűjtésre épülő rendszert készítettem, ahol a módszertani kérdések – például az augmentáció, a mikrofonos domain shift és a végső testelés – kézben tarthatók és védhetően dokumentálhatók.

A saját Log-Mel + 2D CNN modell legfontosabb előnye nem az, hogy minden mérőszámban felülmúlja a transfer learning megközelítést, hanem az, hogy teljes feldolgozási láncként saját kézben van: a hanggyűjtéstől a jellemzőkinyerésen át a valós idejű predikcióig. A YAMNet-alapú összehasonlítás megmutatta, hogy az előtanított embedding erősebb kiindulópont lehet, de a saját modell is értelmezhető, futtatható és a dolgozat céljához illeszkedő megoldást ad. A rendszer tehát nem ipari szintű, általános hangszerfelismerő szolgáltatás-ként értékelendő, hanem egy BSc szintű, végigvezetett mérnöki prototípusként, amelyben a döntések és a korlátok is világosan követhetők.

7.3. Továbbfejlesztési lehetőségek

A rendszerben számtalan továbbfejlesztési lehetőség adott. Elsősorban a kutatás továbbvihető a zajos monofón felismeréstől a tiszta polifón hangszerfelismerés irányába. Ebben az esetben a meglévő softmax aktivációs függvényt sigmoidra kellene cserélnünk a kimeneti rétegen, és többcímkes (multi-label) osztályozást kellene alkalmaznunk, hiszen egyszerre több hangszer is megszólalhat. Ehhez azonban az eddiginél még sokkal több adatra lenne szükség, amelyet szintén az internetről gyűjthet össze az ember, vagy mesterségesen, két különálló hangszer hangját egy szkript segítségével összekeverve állíthat elő.

Ha még nagyobb kihívásra vágyik valaki, el lehet merészkedni a nagy, zajos zenekarakig (orchesztrákig), amelyek a jelen kiváló mérnökeinek is komoly fejtörést okoznak. Minden fejlesztés alapja először úgyis egy stabil és megbízható adathalmaz létrehozása.

Amennyiben a koncepcióval nem a zene felé, hanem egy iparibb környezet irányába indulnánk el, az adathalmaz átszervezésével készíthető egy ipari gépek állapotát felmérő szenzor is. Ez képes lenne megmondani a hang alapján, ha például egy gépből fogyóban van a kenőolaj, ha élettelené vált egy körfűrés (cirkula), vagy egyéb, az ipari gépek állapotával kapcsolatos előrejelzéseket adhatna.

8. IRODALOMJEGYZÉK

- [1] Karol J. Piczak. “ESC: Dataset for Environmental Sound Classification”. *Proceedings of the 23rd ACM International Conference on Multimedia*. 2015, 1015–1018. old.
- [2] Juan J. Bosch, Jordi Janer, Ferdinand Fuhrmann és Perfecto Herrera. “A Comparison of Sound Models for Automatic Instrument Recognition in Polyphonic Music Signals”. *Proceedings of the 13th International Society for Music Information Retrieval Conference (ISMIR)*. 2012, 55–60. old.
- [3] Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan és Mohammad Norouzi. “Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders”. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, 1068–1077. old.
- [4] Carmine-Emanuele Cella, Daniele Ghisi, Philippe Esling és tsai. “TinySOL: An open dataset of orchestra instrument sounds”. *Proceedings of the International Conference on Technologies for Music Notation and Representation (TENOR)*. 2020.
- [5] Vincent Lostanlen, Joakim Andén és Mathieu Lagrange. “Extended Playing Techniques Reconstruction with Scattering Transform”. *Proceedings of the 21st International Conference on Digital Audio Effects (DAFx)*. 2018.
- [6] Emilia Romani, Francesco Reguto, Sergio Giraldo és Rafael Ramirez. “A Real-time System for the Evaluation of Violin Sound Quality”. *Proceedings of the Sound and Music Computing Conference (SMC)*. 2015.
- [7] Brian McFee, Colin Raffel, Dawen Liang, Daniel P Ellis, Matt McVicar, Eric Battenberg és Oriol Nieto. “librosa: Audio and music signal analysis in python”. *Proceedings of the 14th Python in Science Conference*. 2015, 18–25. old.
- [8] Martín Abadi és tsai. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: <https://www.tensorflow.org/>.
- [9] Valerio Velardo. *Audio Signal Processing for Machine Learning*. The Sound of AI. 2020. URL: <https://www.youtube.com/playlist?list=PL-wATfeyAMNqIee7cH3q1bh4QJFAaeNv0> (elérés dátuma 2026. 05. 23.).

9. FÜGGELÉKEK

A függelék az „Automatikus hangszerfelismerés zenefelvételek alapján” című államvizsgaszakdolgozathoz tartozó digitális projektanyag szerkezetét foglalja össze. A cél nem a teljes forráskód bemásolása, hanem annak áttekinthető megmutatása, hogy a nyers hangminták, a kísérleti adathalmazok, a kinyert jellemzők, a betanított modellek és a futtató szkriptek hol találhatóak.

9.1. A projekt könyvtárszerkezete

A benyújtott digitális függelék az alábbi főbb könyvtárakat tartalmazza. A forráskód és a dokumentáció a GitHubon elérhető nyilvános Git adattárban¹ is megtalálható. A nagy méretű, helyben gyűjtött vagy generált adathalmazok és checkpointok a digitális melléklet részei, ezért nem kerültek be a nyilvános adattárba. A számozott szkriptek a feldolgozási csővezeték egymásra épülő lépéseit jelzik.

9.1.1. táblázat: A projekt legfelső szintű könyvtárszerkezete

Könyvtár / Fájl	Leírás
thesis/	A $\text{\LaTeX}$ formátumú szakdolgozat forrásfájljai, stílusfájl, irodalomjegyzék, ábrák és a generált <code>main.pdf</code> .
scripts/	A teljes feldolgozási, jellemzőkinyerési, tanítási és valós idejű felismerési folyamat Python szkriptjei.
owndataset/	A saját gyűjtésű forrásminták és a mikrofonos újrafelvételezéshez kapcsolódó segédanyagok.
dataset_clean/	A tisztított, 1 másodperces clean minták osztályonként rendezett könyvtára.
experiment_datasets/	A négy kísérleti adathalmaz WAV-alapú változata: <code>exp_clean</code> , <code>exp_augmented</code> , <code>exp_naive_deployment</code> és <code>exp_final</code> .
processed_data/	A kinyert NumPy jellemzők és címkék kísérletenkénti almappákban. A clean kísérlet tartalmazza az MFCC, STFT és Log-Mel reprezentációkat, a további kísérletek főként Log-Mel bemenettel futnak; az <code>exp_final</code> a YAMNethez szükséges nyers audio tömböket is tartalmazza.
models/	Az időbélyeggel mentett checkpointok, classification reportok, konfúziós mátrixok és tanulási görbék kísérletenkénti almappákban.
augmented_previews/	Az augmentáció ellenőrzéséhez generált eredeti–augmentált mintapárok és spektrogramábrák.
asd/	A 2. fejezet akusztikai szemléltetéséhez használt kontrollált A4/A3 referenciahangok.

¹<https://github.com/nCsab/Instrument-Recognition>

9.2. A feldolgozási csővezeték szkriptjei

A rendszer teljes feldolgozási lánc indexelt Python szkriptekből áll, amelyek a tervezés sorrendjében futtathatók. Az alábbi táblázat az aktuális, végleges pipeline főbb elemeit foglalja össze.

9.2.1. táblázat: A feldolgozási csővezeték szkriptjei és feladataik

Szkript	Feladat
01_prepare_clean_dataset.py	A saját clean adathalmaz elkészítése: forrásblokkok szeparált felosztása, 1 másodperces szeletelés és zaj/csend minták kezelése.
02_acquire_mic_data.py	A clean minták visszajátszásának, mikrofonos rögzítésének és split/osztály szerinti rendezésének támogatása.
03a_merge_mic_to_train.py	A négy kísérleti adathalmaz összeállítása a clean, augmentált és mikrofonos mintákból.
03b_check_dataset_counts.py	A kísérleti adathalmazok osztályonkénti és split szerinti ellenőrzése.
04a_extract_features.py	A jellemzők kinyerése és normalizálása. A clean baseline esetén MFCC, STFT és Log-Mel, a további kísérleteknél Log-Mel, az exp_final esetén pedig YAMNethez nyers audio is készül.
04b_check_extracted_counts.py	A kinyert NumPy tömbök és címkék méretének ellenőrzése.
05a_realtime_recognition.py	A végső valós idejű Log-Mel + 2D CNN felismerő alkalmazás.
06_train_model_colab/06_train_custom_cnn.py	A saját 2D CNN modell Google Colab környezetben futtatott tanítása, checkpointolása és riportgenerálása.
06_train_model_colab/06_train_yamnet.py	A YAMNet embeddingekre épülő transfer learning referencia tanítása és kiértékelése.
scripts/utils/	Segédfüggvények az augmentációhoz és a jellemzőkinyeréshez.

9.3. Fejlesztői és futtatási környezet

Az alábbi táblázat tartalmazza a projekt fejlesztéséhez és futtatásához szükséges szoftverfüggőségeket.

9.3.1. táblázat: A fejlesztői környezet főbb összetevői

Szoftver / Könyvtár	Verzió	Felhasználás
Python	3.10+	A teljes feldolgozási csővezeték nyelve
TensorFlow / Keras	2.x	A 2D CNN modell építése, tanítása és mentése
Librosa	0.10+	Hangfájlok betöltése, STFT, mel-spektrogram és MFCC kinyerés
NumPy	1.24+	Jellemzőmátrixok tárolása és manipulálása
SoundFile / SoundDevice	–	Hangfájlok olvasása/írása és valós idejű mikrofon-hozzáférés
Matplotlib	3.x	Diagramok, spektrogramok és kiértékelési ábrák készítése
Google Colab	–	GPU-gyorsított modelltanítás felhőkörnyezetben
Git	2.x	Verziókezelés és a projekt előrehaladásának dokumentálása
L ^A T _E X (TeX Live 2026)	–	A szakdolgozat szedése a Sapientia sablon alapján